GenIO: Leveraging LLM Advancements in the Detection, Analysis, and Filling of Gaps During DNA Sequencing

Clara Aparicio Méndez (A20599326)

Research Project Lead: Jaime Cernuda García

Tutors: Anthony Kougkas and Xian-He Sun

August 28, 2025

Contents

List of Figures

List of Tables

Acronym List

LLM *Large Language Model*

RAG *Retrieval-Augmented Generation*

GPU *Graphics Processing Unit*

CPU *Central Processing Unit*

DNA *Deoxyribonucleic Acid*

BPE *Byte Pair Encoding*

MLM *Masked Language Modeling*

CNN *Convolutional Neural Network*

LSTM *Long Short-Term Memory*

RC *Reverse Complementary*

NT *Nucleotide Transformer*

TF-IDF *Term Frequency–Inverse Document Frequency*

RMT *Recurrent Memory Transformer*

GUE *Genome Understanding Evaluation*

LoRA *Low-Rank Adaptation*

GELU *Gaussian Error Linear Unit*

GEGLU *Gated Gaussian Error Linear Unit*

ReLU *Rectified Linear Unit*

MAE-LM *Masked Auto Encoder - Language Model*

BERT *Bidirectional Encoder Representations from Transformers*

BIN *Barcode Index Number*

RoPE *Rotary Positional Embeddings*

RAM *Random Access Memory*

UID *Unique Identifier*

API *Application Programming Interface*

GGUF *GPT-Generated Unified Format Unified Format*

URL *Uniform Resource Locator*

MCP *Model Context Protocol*

AI *Artificial Intelligence*

1. Introduction and objectives

1.1 Introduction

Recent advances in LLMs have revolutionized the way we handle sequence data across domains such as natural language processing, programming, and increasingly, bioinformatics [1]. This project aims to harness the capabilities of LLMs for the specific challenge of **detecting, analyzing, and filling gaps in DNA sequences**, ultimately improving the reconstruction of genetic material. The importance of these models has been recognized at the highest levels, including the awarding of the 2024 Nobel Prize in Chemistry to research centered on LLMs in computational biology [2].

DNA synthesis is fundamental in biology and biotechnology due to its role in the manipulation and study of genetic material. In research, it helps uncover the structure and function of DNA, contributing to the study of genetic diseases and their evolution. It also has applications in industry and agriculture, such as the detection and analysis of bacteria in crops [3].

The process of collecting and reconstructing DNA is complex and faces multiple challenges. It begins with sample collection, where DNA quality can be affected by degradation or contamination. After amplification, it is sequenced, generating a FASTQ file that contains the sequences and their quality scores. Following a cleaning process, the data is converted to FASTA format, which includes only the sequences. These sequences are aligned to a reference genome, fragmented, and gaps are filled to reconstruct poorly sequenced regions. Finally, assembled versions are compared to reference models to estimate the final sequence [4].

This process is prone to errors due to sample quality, DNA synthesizer limitations, and algorithmic constraints. Additionally, the FASTA format is not ideal for handling large-scale data or preserving read quality. Interpreting the interaction between different genome regions also remains challenging, requiring computational models to enhance reconstruction accuracy.

The goal of this project is to optimize and accelerate the DNA reconstruction process through the use of LLMs and advanced data storage and processing techniques. Given that gap-filling models have already been applied in various biological contexts, we are certain that this approach is viable to improve the accuracy and efficiency of genomic sequence reconstruction.

1.2 Objectives

Our proposal addresses the problem of DNA gap filling by exploring advanced approaches such as RAG [5] and autonomous agent-based systems [6]. These techniques allow us to enhance the capabilities of LLMs for domain-specific tasks without modifying their internal parameters. Building on this, we aim to develop a more efficient DNA storage system than the currently used FASTA format. The FASTA format, based on newline-delimited text, lacks efficient indexing structures, making quick and organized access to genetic information difficult.

Vector databases will be used to store and retrieve DNA sequences more efficiently [7]. These databases allow DNA sequences to be represented as numerical vectors, facilitating similarity-based search and comparison. By integrating these databases with RAG techniques, we will be able to generate additional context for LLMs during the sequence reconstruction process, improving the accuracy of predictions.

All components will be integrated into a real-time data streaming pipeline architecture, enabling real-time processing and reconstruction of genomic data. This approach will significantly reduce computation times and improve prediction quality, optimizing a process that is currently slow and computationally expensive.

Additionally, we aim to explore the integration of agent-based components into the pipeline, enabling the system to coordinate multiple reasoning steps and make dynamic use of retrieval, sequence analysis, and decision-making processes to improve gap prediction performance.

This project has an estimated duration of two to three years, with the aim of demonstrating the feasibility of the proposed solution to optimize and accelerate DNA reconstruction. During this period, our focus will be to prove the effectiveness of the approach to justify the need for advanced computational resources, such as high-end GPUs, which are essential for training and deploying large-scale LLMs. By the end of the project, we expect to present results that support the technical and scientific feasibility of the solution, which will allow us to secure funding to continue development and acquire the technological infrastructure needed to bring the project to its final stage.

2. Background

2.1 Overview of Genomic Language Models

Recent breakthroughs in LLMs have opened new opportunities in genomic research, particularly for tasks such as gene prediction, DNA annotation, and sequence reconstruction [1]. These advances are driven by the ability of LLMs to learn complex patterns and long-range dependencies from raw DNA sequences, without requiring extensive manual feature engineering. Inspired by the success of foundation models in natural language processing and computer vision, genomic LLMs are now being developed to capture the intrinsic structure of genomes across diverse species.

Recent work, such as that presented in *Foundation Models in Bioinformatics* [8], emphasizes that genomic LLMs are beginning to demonstrate capabilities beyond those of traditional bioinformatics methods. These models support downstream applications such as variant effect prediction, regulatory region identification, and cross-species transfer learning. Importantly, the transition from fixed k-mer tokenization to more expressive schemes (e.g., byte pair encoding and learned embeddings) has improved the models' representational power and generalization. In addition to model architecture and training scale, tokenization strategy and context length are critical factors influencing the effectiveness of genomic LLMs.

The landscape of genomic language models has rapidly evolved over the past few years, reflecting a convergence of innovations in model architecture, tokenization strategy, and domain specialization. The table below compiles 38 representative models released between 2021 and 2025, covering a broad range of organisms, from human and plants to viruses and archaea.

Architectural Trends: The majority of recent models rely on transformer-based architectures, known for their scalability and ability to model long-range dependencies. While early models such as *GapPredict* and *LookingGlass* used LSTM architectures, recent work has transitioned toward encoder-only (e.g., *DNABERT-2*, *GROVER*), decoder-only (e.g., *DNAGPT*, *LLaMA-Gene*), or hybrid architectures with memory mechanisms (e.g., *GENA-LM*, *Caduceus*).

Tokenization Approaches: Tokenization strategy is one of the most distinguishing factors among these models. Fixed-length k-mer encoding (common in early works like *INHERIT* and *EpigenomicBERT*) is still widely used, especially in models optimized for interpretability. However, newer models increasingly adopt subword techniques such as BPE (e.g., *GROVER*, *DNAGRINDER*) or nucleotide-level encoding (*HyenaDNA*, *megaDNA*) to capture more flexible biological patterns. Some models like *GenomicLLM* also explore

SentencePiece tokenization to accommodate diverse input types.

Context Lengths: Context capacity has steadily increased, with recent models supporting up to 1 Mbp (*HyenaDNA*) or incorporating memory-aware designs that extend effective sequence modeling beyond classical transformer limits (e.g., *GENA-LM*, *Evo*, *Caduceus*). These long-context capabilities are especially important in tasks such as gap-filling to capture long-range dependencies.

Pretraining and Fine-tuning: Almost all modern models are pretrained on large corpora of raw DNA (often multispecies), enabling zero-shot or few-shot transfer to downstream tasks. Many also support fine-tuning, allowing them to adapt to specific sequence types or experimental objectives. Notably, *BarcodeMAE* and *DNABERT-2* introduce novel training protocols to address distribution shifts between masked pretraining and real inference settings.

Species and Domain Focus: While several models target the human genome (e.g., *GROVER*, *Geneformer*, *Caduceus*), others focus on bacteria (*megaDNA*, *Evo*), plants (*AgroNT*, *FloraBERT*), or insects (*BarcodeMAE*). This diversity of sources improves model generalization and supports more equitable genomic research across life sciences.

Overall, the field is moving toward large, adaptable, and biologically grounded models capable of learning both local and global genomic features. The variety in context size, input structure, and model complexity offers a rich space for benchmarking and optimization in specialized genomic tasks such as gap filling.

Descriptor	**Release Date**	**DNA Source**	**Technology**	**Language LLM**	**Fine-tuned/ Pretrained**	**Tokenization**	**Context Length**
BigBird [9]	Jan-21	Human	Transformer	Yes	Yes	BPE	36 Kbp
gapseq [10]	Mar-21	Bacteria	Linear programming	No	No	No	Entire genome
GapPredict [11]	Sep-21	Human	LSTM	No	Yes	one-hot encoding	500 bp
GeneBERT[12]	Oct-21	Human	Encoder-only	Yes	Yes	k-mer	1 Kbp
Enformer [13]	Oct-21	Multispecies	Transformer	Yes	Yes	one-hot encoding	200 Kbp
EpigenomicBERT [14]	Dec-21	Human	Encoder-only	Yes	Yes	k-mer	1000 bp
LookingGlass [15]	May-22	Bacteria/Archaea	LSTM	No	Yes	Nucleotide-level	136 bp
Figbird [16]	Jun-22	Human/Bacteria	Probabilistic EM	No	No	No	Dynamic
ViBE [17]	Jun-22	Virus	Encoder-only	Yes	Yes	k-mer	510 bp
LOGO [18]	Aug-22	Human	Encoder-only	Yes	Yes	k-mer	2 Kbp
FloraBERT [19]	Aug-22	Plant	Encoder-only	Yes	Yes	BPE	1 Kbp
INHERIT [20]	Aug-22	Bacteria/Virus	Encoder-only	Yes	Yes	k-mer	500 bp
GenSLMs [21]	Nov-22	Virus/Bacteria	Decoder-only	Yes	Yes	k-mer	Entire genome
Geneformer [22]	May-23	Human	Encoder-only	Yes	Yes	custom tokenizer	200 Kbp
GPN [23]	Apr-23	Multispecies	CNN	Yes	Yes	Nucleotide-level	512 bp
DNAGPT [24]	Jul-23	Multispecies	Decoder-only	Yes	Yes	k-mer	13.5 Kbp
HyenaDNA [25]	Nov-23	Human	Decoder-only	Yes	Yes	Nucleotide-level	1 Mbp
megaDNA [26]	Oct-23	Bacteria	Decoder-only	Yes	Yes	Nucleotide-level	96 Kbp
GPN-MSA [27]	Oct-23	Multispecies	Transformer	Yes	Yes	Nucleotide-level	128 bp
UTR-LM [28]	Oct-23	Multispecies	Transformer	Yes	Yes	Nucleotide-level	128 bp
DLGapCloser [29]	Aug-24	Fungi/Plant	Bi-LSTM	No	Yes	one-hot encoding	2000 bp
DNABERT-2 [30]	Mar-24	Multispecies	Encoder-only	Yes	Yes	BPE	10 kbp
SegmentNT [31]	Mar-24	Human	Encoder-only	Yes	Yes	k-mer	50 Kbp
Caduceus [32]	Jun-24	Human	BiMamba + RC	Yes	Yes	Nucleotide-level	131 Kbp
AgroNT [33]	Jul-24	Plant	Encoder-only	Yes	Yes	k-mer	6 Kbp

Descriptor	**Release Date**	**DNA Source**	**Technology**	**Language LLM**	**Fine-tuned/ Pretrained**	**Tokenization**	**Context Length**
GROVER [34]	Jul-24	Human	Encoder-only	Yes	Yes	BPE	2.1 Kbp
DNAGRINDER [35]	Sep-24	Multispecies	Encoder-only	Yes	Yes	BPE	12 Kbp
GenomicLLM [36]	Feb-24	Human	Transformer	Yes	Yes	SentencePiece	300 bp
EpiGePT [37]	Dec-24	Multispecies	Encoder-only	Yes	Yes	one-hot encoding	128 Kbp
SpliceBERT [38]	May-24	Multispecies	Encoder-only	Yes	Yes	Nucleotide-level	1024 bp
Evo [39]	Nov-24	Bacteria/Archaea	Transformer	Yes	Yes	Nucleotide-level	131 Kbp
LLaMA-Gene [40]	Nov-24	Multispecies	Decoder-only	Yes	Yes	BPE	300-1000 bp
GENA-LM [41]	Jan-25	Human	Encoder-only	Yes	Yes	BPE	36 Kbp
BarcodeMAE [42]	Feb-25	Insects	Transformer	Yes	Yes	k-mer	768 tokens
Nucleotide Transformer (NT) [43]	Feb-25	Multispecies	Encoder-only	Yes	Yes	k-mer	12 Kbp

Table 2.1: Overview of state of the art genomic language models.

2.2 Model Selection Rationale

From the broad landscape of genomic language models, we selected a subset of the most representative and technically promising models based on several key criteria: novelty, architecture, tokenization strategy, and domain specialization. Our goal was to prioritize recent models that demonstrate strong theoretical advancements and applicability to DNA gap-filling and genome modeling tasks. Below, we present a summary table of the selected DNA language models, including their release dates, tokenization strategies, context sizes, target domains, and underlying architectures.

Model	Date	AR	MLM	Tokenization	Context	Target	Architecture
HyenaDNA	Nov-23	✓	✗	Nucleotide	Long	Human	Decoder
megaDNA	Oct-23	✓	✗	Nucleotide	Long	Bacteria	Decoder
DNABERT-2	Mar-24	✗	✓	BPE	Long	Multispecies	Encoder
GENA-LM	Jan-25	✗	✓	BPE	Long	Human	Encoder
DNAGRINDER	Sep-24	✗	✓	BPE	Long	Multispecies	Encoder
BarcodeMAE	Feb-25	✗	✓	k-mer	Short	Insects	Transformer
GapPredict	Sep-21	✓	✗	One-hot	Short	Human	LSTM
DLGapCloser	Aug-24	✓	✗	One-hot	Short	Fungi/Plant	Bi-LSTM
Figbird	Jun-22	N/A	N/A	No	Short	Human/Bacteria	Probabilistic
NT	Feb-25	✗	✓	k-mer	Long	Multispecies	Encoder
AgroNT	Jul-24	✗	✓	k-mer	Long	Plant	Encoder
SegmentNT	Mar-24	✗	✓	k-mer	Long	Human	Encoder
GROVER	Jul-24	✗	✓	BPE	Long	Human	Encoder
Evo	Nov-24	✓	✗	Nucleotide	Long	Bacteria/Archaea	Transformer
Caduceus	Jun-24	✗	✓	Nucleotide	Long	Human	BiMamba + RC

Table 2.2: Summary of selected DNA language models, tokenization methods, context sizes, target organisms, and architectures.

Among the selected models, **GROVER**, **DNABERT-2**, and **GENA-LM** were chosen as our initial evaluation targets due to their modern transformer-based designs, strong performance in MLM, and the use of BPE, which allows for richer and more adaptive vocabulary representations than fixed k-mer schemes.

GROVER stands out for using next-k-mer prediction and BPE tokenization while being trained exclusively on the human genome. This makes it ideal for interpretability and genome-specific context modeling. **DNABERT-2** builds upon the pioneering DNABERT model, addressing its limitations with improved attention mechanisms (e.g., ALiBi), LoRA for efficient fine-tuning, and FlashAttention for reduced compute time. **GENA-LM** is notable for scaling input lengths up to 36,000 base pairs through a Recurrent Memory Transformer architecture, enabling it to model long-range genomic dependencies with high resolution and minimal memory overhead.

Other models like **Caduceus** and **BarcodeMAE** were included due to their unique innovations: Caduceus incorporates reverse complement (RC) equivariance and bi-directional state-space modeling, while BarcodeMAE eliminates the mismatch between training and inference by pretraining without exposure to `[MASK]` tokens.

The selection also considers domain diversity. For example, **megaDNA** and **HyenaDNA** are designed for bacteriophage and ultralong human sequences respectively, whereas mod-

els like **AgroNT** and **SegmentNT** focus on plant genomes and high-resolution annotation.

We began evaluations with the most recent and architecture-advanced models (2024–2025), emphasizing those with encoder-decoder or memory-enhanced transformer designs, as they are better suited for tasks like gap-filling, which require a strong understanding of sequence context. This approach allows us to assess the practical performance of the model and to highlight the architectural trade-offs in this rapidly evolving field.

The following models have been evaluated and benchmarked so far as part of our initial experimentation phase.

GROVER

GROVER is a DNA language model specifically designed to learn the structure and patterns of the human genome using a transformer-based architecture [34]. It follows a BERT-style design with 12 transformer blocks, each composed of multihead attention mechanisms and feedforward layers, interleaved with normalization steps (LayerNorm). GROVER differentiates from other DNA language models through its approach to tokenize the sequences. While models such as DNABERT and the Nucleotide Transformer rely on fixed-length k-mers to define tokens, GROVER instead applies BPE tokenization method. This allows the model to learn variable-length tokens based on the actual frequency and distribution of sequences, rather than imposing an arbitrary k-mer length.

This type of tokenization allows the model to construct a vocabulary that is more balanced in terms of token frequency, thereby avoiding issues such as Rare Word Problems or training on frequencies instead of on the genome context. Unlike models that incorporate external data or additional species to boost performance, GROVER is trained exclusively on the human genome, ensuring a direct and interpretable link between the learned representations and the sequence itself. This strategy provides a more detailed understanding of genome structure and leverages the strengths of transformer-based architectures, but it remains to be seen how well GROVER performs on our specific task of gap filling.

Intrinsic validation was used to evaluate the model's understanding to the genome, using the metric next-k-mer prediction. This task involves predicting the next token (of a defined length k) in a sequence based solely on the preceding context. This task requires the model to develop a true sense of sequence context and allows fair comparison across models, as it is independent of both the vocabulary size and the underlying architecture.

Results showed that GROVER outperformed other models in the next-6-mer prediction task, achieving 2.0% accuracy, while DNABERT-2 reached 0.6%, and the fixed k-mer models (including NT) remained below 0.4%, regardless of k-mer size. To proof that this improved performance was due to better contextual understanding rather than frequency biases in the genome, the model was compared with TF-IDF models, which rely solely on token frequency statistics to make predictions. While the best-performing TF-IDF model reached 1.1% accuracy for next-6-mer prediction, it still underperformed compared to GROVER. This confirmed that GROVER's performance is not simply a reflection of token frequency, but rather an indication that it has effectively learned the underlying structure and dependencies in the genome sequence. This analysis supports GROVER's strong ability to capture sequence context in an unbiased, generalizable way, which is essential for tasks like gap filling.

GROVER was trained using a masked token prediction objective, in which the model learns to infer missing tokens within a DNA sequence based on surrounding context. In the original study, GROVER achieved a 21% accuracy when limited to predicting only the most likely token (top-1). However, when the evaluation criterion was relaxed to consider a prediction successful if the correct token appeared among the top 60 candidates (approximately 10% of the total vocabulary) the accuracy improved substantially to 75%. These results reflect the model's ability to assign high probability to biologically plausible tokens, even if the top prediction is not always correct.

DNABERT-2

DNABERT-2 is a DNA foundation model developed to address the limitations of the original DNABERT, introduced in 2021 [30]. While DNABERT was pioneering in modeling DNA sequences using transformer encoder architectures similar to BERT, it faced several challenges that DNABERT-2 aims to resolve.

One of the primary limitations of DNABERT was its use of k-mer tokenization, which splits sequences into overlapping fixed-length substrings. Although useful in some contexts, this approach introduces redundancy and information leakage, particularly when processing highly similar sequences. DNABERT-2 replaces this with BPE, to learn meaningful subword patterns, resulting in more compact sequence representations and improved training efficiency.

Another significant difference is that DNABERT was that it was pretrained exclusively on the human reference genome, which restricted its ability to generalize to broader genomic contexts. In contrast, DNABERT-2 is trained on genomes from multiple species, enabling it to capture a more diverse range of genomic patterns and to perform better across a variety of tasks and organisms.

A technical bottleneck in DNABERT was its 512-token input length limit, imposed by learned positional embeddings. DNABERT-2 eliminates this constraint by adopting ALiBi (Attention with Linear Biases), a positional encoding strategy that encodes relative positions directly into attention scores using fixed, non-learned biases. Unlike traditional methods such as sinusoidal, learned, or rotary embeddings, ALiBi allows the model to generalize to sequences of arbitrary length, making it particularly well-suited for genomic tasks involving long-range interactions.

To further improve training efficiency, DNABERT-2 integrates FlashAttention, a memory-optimized attention mechanism that reduces computational overhead. These architectural upgrades make DNABERT-2 up to 21× smaller and roughly 92× faster in GPU time compared to models like the Nucleotide Transformer, while achieving competitive performance.

The creators of DNABERT-2 also introduced the Genome Understanding Evaluation (GUE and GUE+) benchmark, consisting of 36 datasets spanning 9 genome-related tasks, including promoter prediction, gene expression modeling, and chromatin state classification. This benchmark provides a fair and consistent framework for evaluating genomic foundation models.

In terms of fine-tuning, DNABERT-2 incorporates Low-Rank Adaptation (LoRA), a parameter-efficient method that updates only a low-rank decomposition of the weight

matrices. This drastically reduces memory usage and the number of trainable parameters, allowing effective fine-tuning even on consumer-grade GPUs.

Finally, DNABERT-2 replaces the standard ReLU activation function with GEGLU (Gated GELU), which combines a gating mechanism with a smoother GELU activation function. This enables the model to better capture complex, non-linear dependencies within genomic sequences, leading to more expressive and biologically relevant representations.

GENA-LM

The GENA-LM project introduces a suite of open-source, transformer-based DNA language models specifically designed to overcome a central challenge in genomics: capturing long-range dependencies across tens of thousands of base pairs [41]. While earlier models such as DNABERT laid the groundwork for interpreting genomic sequences, they remain limited to relatively short input lengths (typically under 4 kilobases). GENA-LM addresses this constraint by supporting inputs of up to 36,000 base pairs, without compromising computational efficiency.

To enable modeling of long sequences, GENA-LM integrates a Recurrent Memory Transformer (RMT) architecture. This mechanism allows sequences to be split into 512-token segments (approximately 4.5 kb each), with memory tokens transferring contextual information from one segment to the next. These memory tokens are trainable and updated via backpropagation, effectively transforming the transformer into a recurrent unit that avoids the quadratic time and memory complexity of standard self-attention.

During fine-tuning, task-specific strategies were applied:

- For promoter prediction, only the final segment was used to compute the loss.
- For splice site prediction, loss was computed across all segments.
- Optimization was carried out using the AdamW optimizer with learning rates between 10^{-4} and 2×10^{-5}, along with early stopping.

In our experiments, we employed several models from the same family. First, the `gena-lm-bert-base-t2t` variant, which is based on a 12-layer BERT architecture with a maximum input length of 512 tokens, corresponding to approximately 4,500 base pairs due to its k-mer-aware tokenization. We also experimented with the `bigbird-base- t2t` model, which shares the 12-layer BERT architecture but allows for a much longer input length of 4,096 tokens, covering around 36,000 base pairs. They both use BPE with a 32,000-token vocabulary initialized from the nucleotide alphabet (A, T, C, G, N). The tokenizer was trained on a mixed corpus including the T2T v2 human genome, the 1000 Genomes Project, and a diverse set of multi-species genomes. Special tokens such as `[CLS]`, `[SEP]`, `[PAD]`, `[UNK]`, and `[MASK]` are included, and stretches of more than 10 consecutive unknown nucleotides (Ns) are collapsed into a single "–" token during preprocessing.

By comparing models with different input capacities (512 tokens for `gena-lm-bert-base-t2t` and 4,096 tokens for `bigbird-base-t2t`) we can assess the trade-offs between contextual range and computational efficiency, and better understand how sequence length impacts model performance on gap-filling tasks.

The original GENA-LM paper evaluated model variants using the MLM task, a self-

supervised learning approach where models predict masked tokens based on surrounding context. Training was conducted on the T2T human genome assembly and augmented with SNP data from the 1000 Genomes Project and sequences from a broad set of eukaryotic species. The authors found that sparse attention mechanisms were particularly beneficial for handling long sequences (up to 36 kb). However, they also acknowledged that high MLM accuracy does not always translate into superior performance on downstream biological tasks, underscoring the importance of task-specific evaluation.

Nucleotide Transformer

The Nucleotide Transformer addresses a key limitation found in earlier genomic models: reliance on a single reference genome. Instead, it was pretrained on a diverse dataset of over 3,200 phased human genomes from the 1000 Genomes Project and 850 genomes from other species, including both model and understudied organisms. This extensive training enables the model to capture a wide range of genetic variation and enhances its ability to generalize across organisms and genomic contexts [43].

Nucleotide Transformer adopts a BERT-style transformer architecture, with a tokenization strategy based on 6-mer sequences, backed by single-nucleotide tokens when needed. The tokenizer has a vocabulary of 4,105 tokens and supports input sequences of up to 1,000 tokens. The model is trained using a classical MLM objective: 15% of tokens are selected for prediction, with 80% replaced by a `[MASK]` token, 10% by a random token, and 10% left unchanged.

Several variants of the model have been released, ranging from lighter versions like the `v2-50M` and `v2-100M` to large-scale models like `nucleotide-transformer-2.5b-multi-species` and `500m-1000g`, which were trained on extensive datasets including human genetic variation and multi-species genomes. These variants allow the model to be deployed in a variety of computational environments and research contexts. Our experiments focused on `nucleotide-transformer-2.5b-multi-species` and `nucleotidetransformer-v2-250m-multi-species`.

Nucleotide Transformer has demonstrated superior performance over specialized methods in a range of genomic prediction tasks, including *variant prioritization*, *molecular phenotype prediction*, and *functional element discovery*. Its ability to learn from raw DNA without explicit supervision makes it particularly valuable in settings with limited labeled data, where contextual understanding is essential.

This model was selected for our evaluation not only due to its strong empirical performance, but also because of its broad training data, effective pretraining strategy, and open-source availability. Its direct handling of genomic context and scalable architecture make it a highly capable foundation model for sequence completion.

AgroNT

AgroNT is a foundational DNA language model trained on genomic sequences from 48 plant species, with a focus on crop genomes [33]. It builds upon the NT architecture, representing a domain-specific fine-tuning of NT for plant genomics. The model is based on transformer architectures and uses MLM to learn regulatory and structural patterns across a broad phylogenetic range. Tokenization is performed using non-overlapping 6-mers, resulting in a vocabulary of 4,104 tokens, including all canonical 6-mer combinations

from the nucleotide alphabet, special symbols (e.g., `[PAD]`, `[MASK]`, `[CLS]`), and singleton tokens for ambiguous bases like `N`. This setup allows AgroNT to process sequences as long as 1,024 tokens (approximately 6,000 base pairs), striking a balance between biological resolution and computational feasibility.

Unlike models constrained to a single reference genome or organism, AgroNT leverages multi-species training data, improving generalization and robustness across plant taxa. Additionally, its architecture incorporates parameter-efficient fine-tuning via IA^3, which allows downstream adaptation with minimal updates to the base model. Notably, it has shown strong performance in regulatory element identification (e.g., splice sites, APA signals, enhancers), expression level prediction, and zero-shot variant effect estimation. These capabilities make it particularly valuable for tasks such as gap filling and clustering in species with limited annotations.

We selected AgroNT for our experiments due to its high performance on long sequences, biologically informed tokenization scheme, and transfer learning capabilities.

BarcodeMAE

MLM is one of the most commonly used pretraining strategies for DNA language models. Most models adopt a masking scheme similar to that of the original BERT, in which 80% of selected tokens are replaced with `[MASK]`, 10% remain unchanged, and the remaining 10% are substituted with random tokens.

A fundamental issue with this approach is the distribution shift between training and inference phases. During training, the model is exposed to synthetic inputs containing `[MASK]` tokens, learning to predict them based on surrounding context. However, at inference time, the model is applied to fully visible sequences that contain no `[MASK]` tokens. This mismatch causes the model to become overly specialized in masked token prediction, rather than learning useful representations for real DNA inputs. As a result, models trained under standard MLM objectives may be inefficient in deployment and underperform in downstream tasks.

BarcodeMAE addresses this limitation by adapting the MAE-LM architecture to DNA sequence modeling. In this model, the encoder never sees `[MASK]` tokens; instead, only the decoder is responsible for reconstructing the masked segments. The architecture consists of a symmetric encoder-decoder design, with each component comprising 6 transformer layers and 6 attention heads. After pretraining, the decoder is discarded, and only the encoder is retained for downstream tasks. This design ensures that the encoder is trained under realistic inference conditions, improving generalization and representation quality.

BarcodeMAE was evaluated using three core experiments. In a closed-world task, the model was tested on genus-level classification of unseen species using a 1-nearest-neighbor approach with cosine similarity. BarcodeMAE achieved 69.0% accuracy, over 10% higher than BarcodeBERT, demonstrating improved generalization to novel species within known taxonomic groups.

In an open-world evaluation, a BIN reconstruction task was performed using zero-shot clustering. While DNABERT-S obtained the highest clustering score, BarcodeMAE delivered competitive performance and achieved the best overall results when combining closed- and open-world scores using harmonic mean.

An ablation study further explored architectural configurations, including encoder/decoder depth, attention heads, and k-mer sizes. Interestingly, unlike typical NLP findings favoring shallow decoders, deeper and more balanced architectures performed better in genomic settings.

The authors also studied `[MASK]` token dependency and found that models like BarcodeBERT heavily rely on masked tokens during inference, which impairs generalization. In contrast, BarcodeMAE provides more realistic and efficient representations since the encoder is trained on unmasked inputs.

Despite these promising results, an important limitation remains: the source code for BarcodeMAE has not been released. This hinders reproducibility and practical adoption of the model in independent research settings.

Caduceus

DNA modeling presents unique challenges compared to language processing. First, non-coding regions play a critical role in gene regulation and cellular behavior. Second, DNA is bi-directional: upstream or downstream elements may influence gene activity, meaning that any model must understand sequence context in both directions. Third, DNA exists as two strands that are reverse complements of one another, encoding the same information in opposite directions. Accurately modeling this reverse complement (RC) property is essential for biological fidelity.

Additionally, some regulatory mechanisms operate over very long genomic distances, making it necessary for models to capture long-range dependencies. Traditional transformer architectures struggle with this due to their quadratic scaling in memory and compute.

To address these problems, the authors of Caduceus introduced new components based on the Mamba architecture, which efficiently handles long sequences without relying on attention mechanisms. They propose two new modules:

- **BiMamba** introduces bi-directionality to the architecture, allowing context to be gathered in both forward and backward directions.
- **MambaDNA** adds RC-equivariance, enabling the model to treat both DNA strands as informationally equivalent.

These innovations are combined into a model family called Caduceus, which is the first to support both long-range modeling and RC equivariant language modeling for genomic data.

Unfortunately, due to unresolved compatibility issues on our system, we have not yet been able to properly test the Caduceus model in practice. Troubleshooting its installation and environment setup is currently ongoing.

Evo

Evo is a genomic foundation model designed to enable both prediction and generation tasks across the molecular, systems, and genome [25]. Unlike previous models limited to specific modalities or short sequences, Evo is trained as a large autoregressive model with 7 billion parameters and a context length of up to 131 kilobases. It operates at single-

nucleotide, byte-level resolution, allowing it to capture fine-grained biological patterns directly from raw DNA.

The model is based on the StripedHyena architecture, which combines data-controlled convolutional operators (Hyena layers) with a small number of multi-head attention layers. This hybrid structure enables efficient long-range sequence modeling without the quadratic complexity of traditional transformers. Evo includes 29 Hyena layers and 3 attention layers with rotary positional embeddings (RoPE). This design allows Evo to process very long sequences and scale favorably in terms of compute efficiency and performance.

Training was conducted on the OpenGenome dataset, comprising 300 billion nucleotide tokens from over 80,000 bacterial and archaeal genomes, as well as millions of phage and plasmid sequences. The training procedure was performed in two stages: an initial phase with an 8,192-token context, and a second phase extending to the full 131,072-token context. Evo uses a next-token prediction objective (autoregressive learning), enabling it to model DNA generatively.

Evo demonstrates strong zero-shot performance in predicting mutation effects in proteins and noncoding RNAs, identifying essential genes, and generating biologically plausible sequences at scales up to 650 kilobases.

We initially intended to include Evo in our experiments due to its strong performance, single-nucleotide resolution, and ability to model long genomic sequences. However, the model relies on FlashAttention for computational efficiency, which is currently incompatible with CUDA version 12.5, the default runtime in Google Colab. This limitation made it infeasible to run Evo within our computational environment.

3. Model Evaluation

In this chapter, we describe the methodology followed to evaluate the selected genomic language models. First, we present the dataset used to assess the models' ability to predict DNA sequences in a gap-filling setting. The dataset was carefully designed to cover a broad range of bacterial diversity and sequence lengths.

Next, we outline the evaluation protocol and metrics used to quantify model performance, such as exact match accuracy and top-k token prediction accuracy. These metrics allow us to compare different model architectures under consistent and biologically meaningful criteria.

All experiments were initially conducted locally on a standard computing setup featuring an 11th Gen Intel Core i7-1165G7 @ 2.80GHz CPU with 16 GB of RAM. While no high-performance GPUs were used during this phase, the experimental design ensured that results remained reproducible and comparable across models, independent of hardware constraints.

Subsequently, a processing pipeline was developed for MLM models, and the ones not restricted by hardware limitations were migrated to run on a Google Colab T4 GPU. The only model that continued to run on CPU was DNABERT-2.

3.1 Dataset

To evaluate the performance of the selected genomic language models on DNA gap-filling tasks, we used a custom-built benchmark dataset specifically designed to test sequence reconstruction accuracy. This dataset was generated to span a broad taxonomic range, covering all 73 bacterial phyla available in the NCBI database with complete genome data. In total, the dataset comprises 381 genomes and contains 38,100 test files, evenly distributed across four sequence lengths: 1 kbp, 2 kbp, 5 kbp, and 10 kbp.

Each file in the dataset is a plain-text document containing both metadata and sequence data. It includes:

- **Accession**: the GenBank identifier of the genome sequence from which the example was extracted.
- **Position**: the genomic coordinates of the sequence segment (in base pairs).
- **Context length**: the number of base pairs provided to the model as input.
- **Target length**: the number of base pairs the model is expected to reconstruct.
- **CONTEXT**: the input sequence fragment (left side of the gap).

- **TARGET**: the ground-truth continuation that the model must predict.

This format facilitates consistent evaluation across all models, allowing token-by-token prediction and comparison against ground truth data.

An example of a test file is shown below:

```
# Accession: AP012051.1
# Position: 48069-50069
# Context length: 1000 bp
# Target length: 1000 bp

CONTEXT:
TTTCTAATTCAGACTAG...

TARGET:
GGGGGAAATGTACAATA...
```

3.2 Benchmarking of Models

To assess the performance of the models on a realistic sequence prediction task, we implemented a token-by-token gap filling evaluation. This task simulates the models' ability to reconstruct a missing segment of DNA (the target) based on a provided context sequence. The evaluation is designed to measure how effectively each model captures sequence context and prioritizes biologically plausible continuations.

The model receives the context as input (from the test file) and attempts to iteratively predict the continuation (the target), filling in one token at a time.

In each iteration:

- The model appends a `[MASK]` token to the current sequence.
- It predicts the top 60 most probable tokens to fill the mask, using its internal BPE vocabulary.
- If any of the top 60 predicted tokens match the prefix of the remaining target sequence, the prediction is considered correct and the matching token is appended to the sequence.
- Otherwise, the top-1 prediction is appended instead, and the step is marked as incorrect.

This method follows the evaluation protocol described in the original GROVER paper [26], using a top-60 threshold to assess whether the correct token appears among the top 10% of the vocabulary.

Two accuracy metrics were computed:

1. **Top-60 token accuracy**: the proportion of prediction steps where the correct token was found among the top 60 predictions.
2. **Exact match accuracy**: calculated at multiple completion percentages (10%, 25%, 50%, 75%, and 100%) by comparing the predicted sequence against the ground truth at the nucleotide level.

In addition to these accuracy-based evaluations, we employed standard classification metrics (precision, recall, and F1 score) to capture a more nuanced understanding of sequence-level prediction quality. These metrics were computed as follows:

- For each test sequence, the predicted and target strings were aligned character by character (nucleotide by nucleotide).
- The predicted sequence was truncated or padded as needed to match the length of the ground truth up to the evaluation coverage (e.g., 10%, 25%, etc.).
- Each character (A, T, C, G, or other valid nucleotide symbols) was treated as a discrete class label.
- A classification report was generated by comparing the predicted class labels to the true class labels at each position.
- We used `macro-averaging`, which calculates the metric independently for each class and then takes the average, treating all classes equally regardless of their frequency.

Precision measures the proportion of predicted nucleotides that are actually correct, thus reflecting the model's tendency to make false positive predictions. **Recall** measures the proportion of correct nucleotides that were successfully predicted, indicating how well the model captures relevant sequence information. The **F1 score**, being the harmonic mean of precision and recall, balances these two aspects and serves as a robust indicator of overall prediction quality.

This classification-based evaluation framework provides a complementary perspective to token-level accuracy metrics, enabling a more detailed analysis of prediction behavior, particularly in identifying systematic biases toward or against specific nucleotide classes. It also supports model comparisons across different genomic contexts and sequence lengths, offering insights into the strengths and weaknesses of each architecture under a unified evaluation protocol.

The accuracy and classification metrics provide a detailed view on each model's ability to generalize genomic structure and offers insights into how well its vocabulary and transformer architecture capture biological patterns in an unsupervised setting.

We further evaluated the models along the following auxiliary dimensions to assess their usability and computational performance:

- **Levenshtein Distance:** this metric quantifies the minimum number of single-character edits (insertions, deletions, or substitutions) required to transform the predicted sequence into the ground truth [44]. A lower Levenshtein distance indicates higher sequence similarity and prediction fidelity.
- **Disk Usage:** disk consumption was monitored during model inference to evaluate resource efficiency. The measurements reflect the additional disk space used during the processing of each input file, capturing temporary data and intermediate outputs.
- **RAM Usage:** we recorded the peak memory usage (RAM) incurred during each prediction to assess memory efficiency. This is crucial for understanding the computational footprint of each model and identifying those better suited for resource-constrained environments.

- **File Processing Time:** we measured the wall-clock time required to process each sequence, including loading, tokenization, prediction, and output writing. This metric reflects the overall computational efficiency and responsiveness of each model in a real-world setting.

Together, these metrics offer a comprehensive benchmarking framework that combines predictive accuracy with practical considerations of computational cost and resource consumption. This approach enables a balanced evaluation of model performance, scalability, and deployment viability across diverse genomic prediction tasks.

3.3 Model Evaluation Results

3.3.1 Top-60 Accuracy

Top-60 Accuracy is a custom evaluation metric used in this study to measure the model's ability to predict the correct next token within its top 60 most probable outputs. Unlike standard accuracy, which only considers the top-1 prediction, Top-60 Accuracy provides a broader perspective on the model's capacity to prioritize the correct token among a larger set of plausible alternatives. This is particularly useful in biological sequence modeling tasks, where the vocabulary is large, multiple biologically plausible next tokens may exist, and the top prediction might be slightly off, even though the correct answer is still ranked nearby.

In this evaluation, the metric was computed step-by-step throughout the prediction of the target sequence. The final reported value represents the proportion of steps in which the correct token was included among the model's top 60 predictions.

Below, we present the Top-60 Accuracy results obtained for each model, evaluated at multiple coverage thresholds (10%, 25%, 50%, 75%, and 100%) and across different input context lengths (1000 bp, 2000 bp, and 5000 bp).

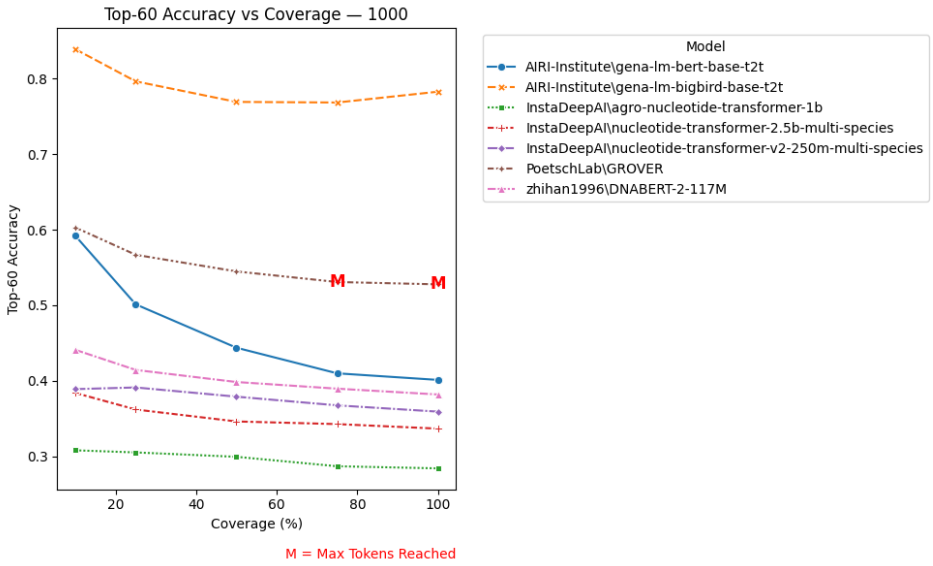

Figure 3.1: Top-60 Accuracy vs Coverage with a context length of 1000 bp.

Among the models evaluated, for a 1000 bp context length, GROVER consistently achieves the second highest Top-60 Accuracy scores, starting at approximately 0.60 for 10% cov-

erage and gradually declining to just above 0.53 at full sequence coverage. This modest drop in performance suggests a high degree of robustness across increasing input lengths. However, it is important to note that GROVER reaches its maximum input token capacity at 75% and 100% coverage. This constraint is indicated by the red "M" markers on the plot, which signal the points at which the model is no longer able to process the entire input sequence due to its architectural limits. Although GROVER maintains strong predictive performance at these higher coverage levels, the reliability of these results is uncertain, as the model is operating beyond its designed capacity.

The `gena-lm-bert-base-t2t` model also performs strongly at the early stages, with an initial Top-60 Accuracy of around 0.59. However, its performance declines more sharply than GROVER's, falling to approximately 0.40 by the time the entire target sequence is predicted. This suggests that while the model is well-suited for short-range predictions, it struggles with longer contexts, possibly due to an accumulation of noise as the input grows.

DNABERT-2 delivers moderate and relatively stable performance, beginning with an accuracy of about 0.44 and ending around 0.39. Unlike GROVER, this model does not reach token limitations during prediction and shows a smooth, gradual decline in performance with increased coverage.

Other models, including `nucleotide-transformer-v2-250m` and `nucleotide-transformer-2.5b-multi-species`, exhibit similar trends of declining accuracy with increasing sequence length. The 2.5B variant, despite being significantly larger, does not outperform its smaller counterparts, with scores ranging from approximately 0.37 to 0.34. This may indicate inefficiencies in applying such a large model to step-wise nucleotide generation or difficulties in generalizing across context windows.

`Agro-nucleotide-transformer-1b` shows the lowest Top-60 Accuracy overall, beginning near 0.31 and decreasing slightly across the evaluated coverage thresholds. This model may be less optimized for token-by-token DNA sequence prediction tasks.

We can observe that `gena-lm-bigbird-base-t2t` introduces a notable improvement in performance. This model outperforms all others, starting at an impressive Top-60 Accuracy of approximately 0.85 at 10% coverage. Although its accuracy slightly decreases as coverage increases, dipping to around 0.78 at full coverage, it still maintains a clear lead over the rest.

With a context length of 2000 base pairs, the `gena-lm-bert-base-t2t` model initially achieves one of the highest Top-60 Accuracy, reaching approximately 0.52 at 10% coverage. However, as the prediction progresses and a larger portion of the target sequence is generated, its performance declines substantially, falling to around 0.40 at full coverage. Additionally, the model avoids reaching its maximum token limit until the 75% coverage threshold.

In contrast, GROVER maintains a flat Top-60 Accuracy line (~ 0.395 across all coverage levels), and each of its evaluation points is flagged as having reached the maximum number of allowable tokens. This suggests that GROVER is unable to handle 2000 bp long sequences as input.

DNABERT-2, though slightly better than GROVER in early stages (~ 0.435 at 10%), exhibits a steady decline in accuracy to the same 0.395 plateau by 100% coverage. Like GROVER, it also hits token limits at early (25%) coverage levels. The convergence in per-

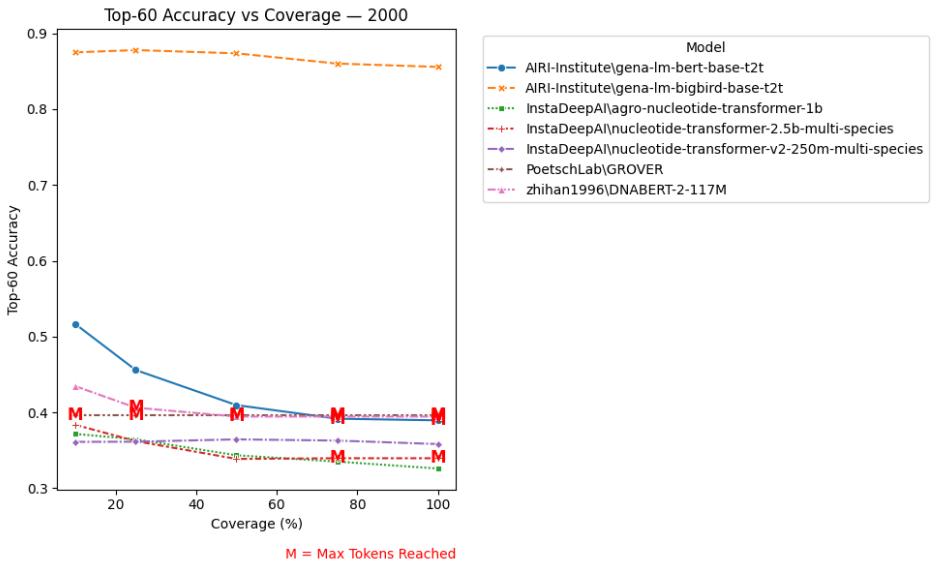

Figure 3.2: Top-60 Accuracy vs Coverage with a context length of 2000 bp.

formance at longer coverage might indicate that, for these models, sequence continuation beyond their max input size becomes unreliable or impossible.

The larger `nucleotide-transformer-2.5b-multi-species` model performs worse than expected, with Top-60 Accuracy values starting around 0.38 and flattening to $\sim$ 0.34. Its smaller variant, the `nucleotide-transformer-v2-250m`, outperforms it, maintaining more stable accuracy around 0.36-0.37. This implies that the added model capacity of the 2.5B version does not translate into improved step-wise prediction for long sequences, possibly due to inefficient utilization or overfitting.

The `agro-nucleotide-transformer-1b` again ranks lowest in Top-60 Accuracy, starting at $\sim$ 0.375 and dropping gradually to $\sim$ 0.325, confirming its general underperformance across both short and long contexts.

`Gena-lm-bigbird-base-t2t` stands out as the top performer, maintaining exceptional and stable Top-60 Accuracy above 0.85 across all coverage levels. Its robustness to input length highlights its strength in modeling long-range dependencies. In contrast, `gena-lm-bert-base-t2t` starts strong but its accuracy declines sharply, dropping to around 0.40 at full coverage, suggesting it is more suited to shorter-range predictions. GROVER and DNABERT-2 maintain consistent performance early on, but are limited by maximum token capacity, truncating beyond 10–25% coverage. Meanwhile, the nucleotide transformer models exhibit moderate, relatively flat performance, with larger model sizes offering no clear advantage. Overall, none of the models completely overcome the challenges of long-context genomic prediction at this sequence length, with the exception of `gena-lm-bigbird-base-t2t`, which remains robust and accurate throughout.

For 5000 bp input sequences, models like GROVER, `gena-lm-bert-base-t2t`, and DNA BERT-2, which previously delivered strong or stable performance at shorter lengths, are now entirely incapable of producing usable predictions. Their Top-60 Accuracy drops to 0.0 at every coverage level, clearly indicating that token input limitations prevent them from generating any valid outputs at this context length.

In contrast, `agro-nucleotide-transformer-1b` and `nucleotide-transformer-v2-250m`

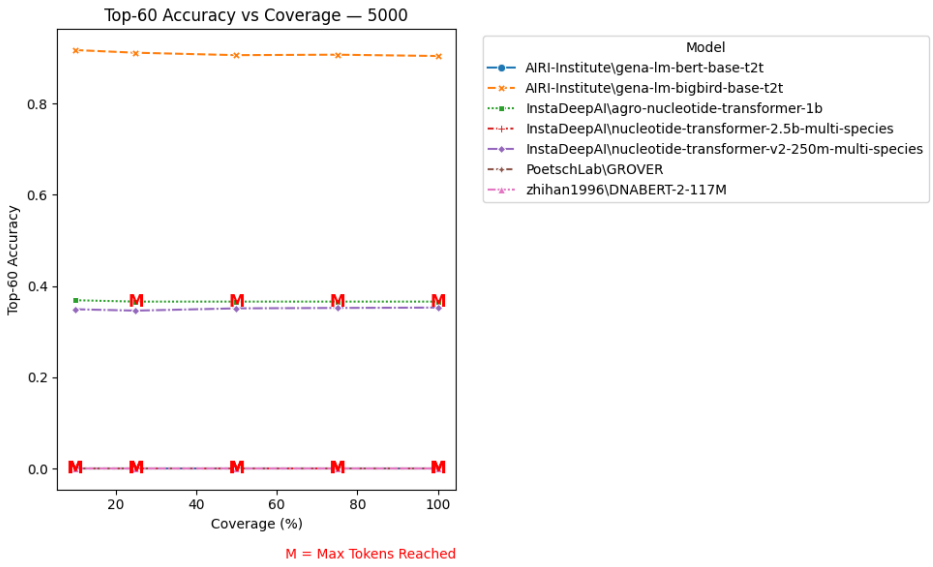

Figure 3.3: Top-60 Accuracy vs Coverage with a context length of 5000 bp.

maintain non-zero accuracy across the full range of coverage, with performance hovering around 0.36 and 0.87, respectively. However, `agro-nucleotide-transformer-1b` hits the maximum token threshold at 25% coverage.

The standout performer is once again `gena-lm-bigbird-base-t2t`, which continues to deliver exceptional accuracy above 0.87 across all coverage levels, without any sign of token constraint. This reinforces its capacity to manage long-range dependencies, making it unique in extended genomic contexts.

3.3.2 Precision

Precision measures the proportion of correctly predicted tokens among all tokens predicted as relevant. In the context of sequence modeling, high precision indicates that the model is confident and accurate in its predictions, minimizing false positives. Below, we present the precision results obtained across different models, coverage levels, and input sequence lengths (1000 bp, 2000 bp, and 5000 bp).

For 1000 bp input sequences, `gena-lm-bigbird-base-t2t` leads all models with the highest precision, starting at approximately 0.87 at 10% coverage and maintaining values consistently above 0.85 throughout. This outstanding and stable performance highlights its ability to make precise predictions even as the generation length increases.

GROVER also performs strongly, beginning near 0.65 and declining gradually to 0.58 by full coverage. However, this decline is associated with reaching its maximum token capacity at higher coverage levels, suggesting its performance is likely bottlenecked by input constraints. `gena-lm-bert-base-t2t` shows a similar initial strength in precision (0.645) but experiences a more significant decrease to about 0.48, indicating greater sensitivity to context length and potential struggles with long-range coherence.

Meanwhile, DNABERT-2, `nucleotide-transformer-2.5b-multi-species` and `nucleotide-transformer-v2-250m` exhibit moderate precision, with relatively flat or slightly improving trends across coverage levels. This stability may point to a conservative generation strategy, yielding moderate precision but limited adaptability to longer contexts.

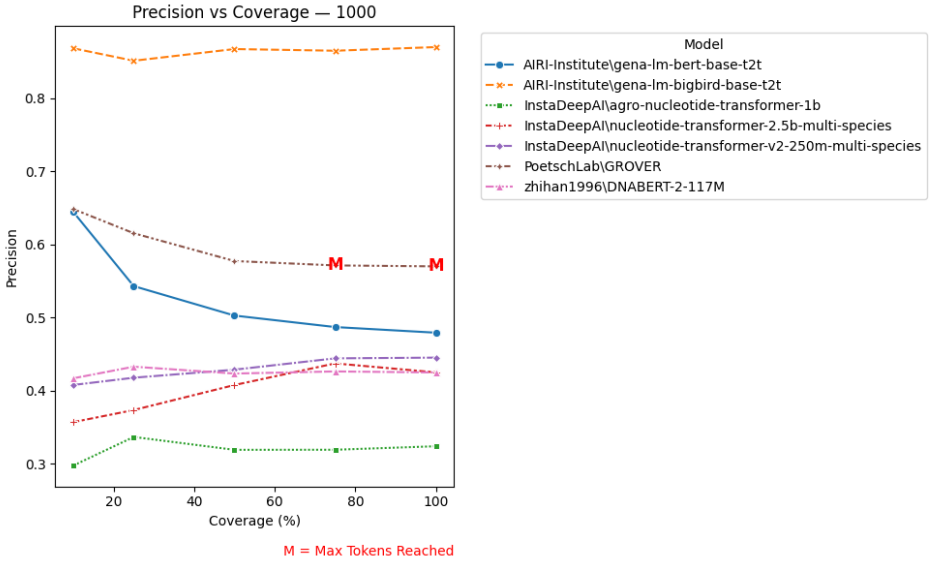

Figure 3.4: Precision vs Coverage with a context length of 1000 bp.

Finally, `agro-nucleotide-transformer-1b` performs the weakest in terms of precision, fluctuating between 0.30 and 0.34, revealing its challenges in consistently generating accurate predictions regardless of coverage.

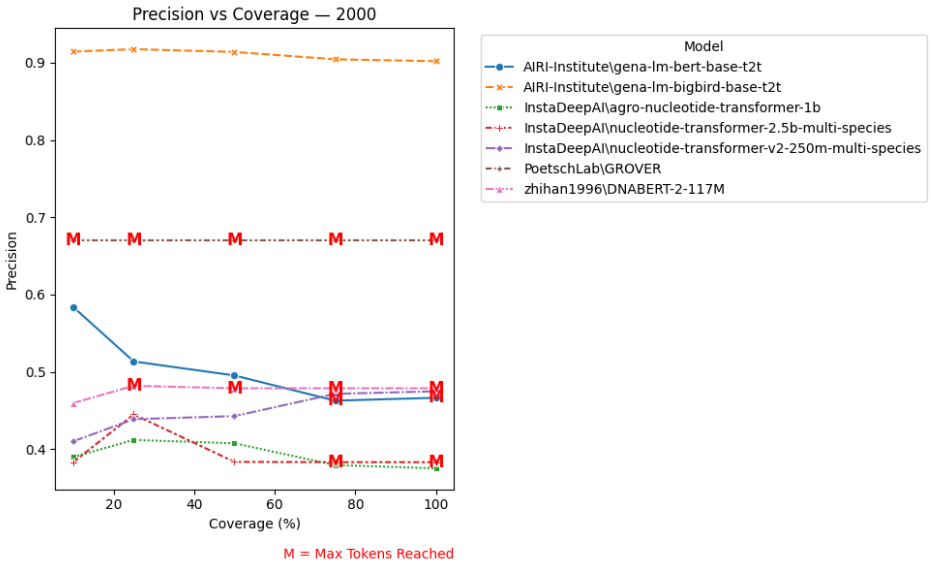

Figure 3.5: Precision vs Coverage with a context length of 2000 bp.

For 2000 base pairs input sequences there is a more constrained landscape due to token limitations, particularly affecting models like GROVER and DNABERT-2. GROVER maintains a flat precision score of approximately 0.67 across all coverage levels. However, in each of its plotted points the model reached its maximum token capacity, indicating that this high performance is not real.

The model `gena-lm-bert-base-t2t` begins with strong precision (~ 0.58 at 10% coverage), but experiences a consistent decline as coverage increases, dipping below 0.47 by 100%. This trend suggests that while the model can make accurate predictions in the early stages, its performance is increasingly affected as more of the sequence is generated.

Also, the model reaches its maximum input token capacity at 75% coverage.

DNABERT-2 performs steadily throughout, showing a modest precision (~ 0.475) without sharp drops. It also hits the token limit from 25% onward, which preserved stability but restricted its potential for improvement.

Other models, such as `nucleotide-transformer-2.5b-multi-species` and textttagro-nucleotide-transformer-1b, exhibit lower precision values ($\sim 0.38-0.44$) and relatively flat or slightly declining trends. Their limited precision indicates less effective token selection, which remains consistent regardless of how much of the target sequence is being predicted.

On the other hand, the `nucleotide-transformer-v2-250m-multi-species` model exhibits an upward trend, with values rising from 0.41 to 0.47 as sequence coverage increases. Unlike many other models, it does not hit input size limitations within the evaluated range, suggesting that it is capable of leveraging additional context effectively. This behavior indicates improved performance with longer input sequences.

Lastly, `gena-lm-bigbird-base-t2t`, on the other hand, stands out with exceptional precision, consistently above 0.90 across all coverage levels. Unlike other models, it maintains this high accuracy without being affected by input size limitations, confirming its capability to handle long-range dependencies effectively and leverage broader context.

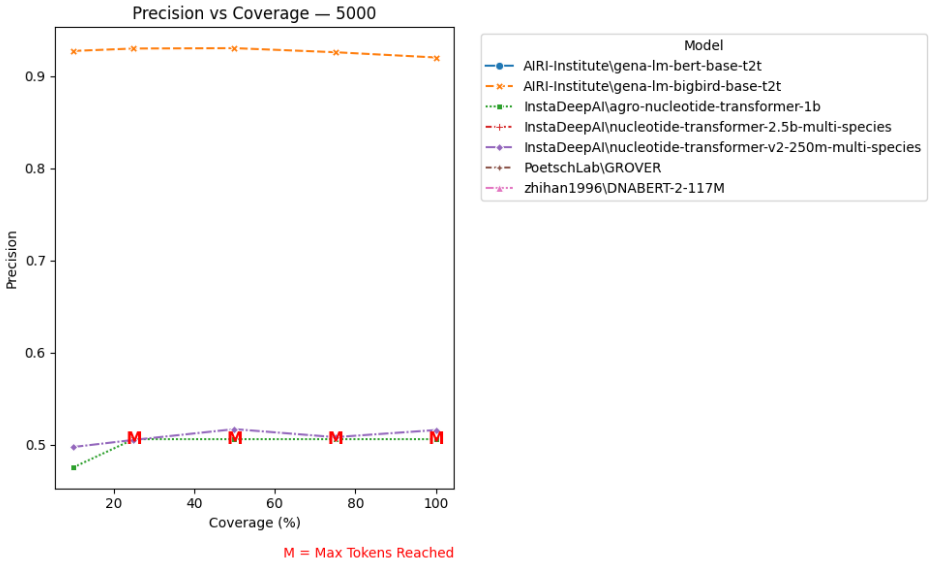

Figure 3.6: Precision vs Coverage with a context length of 5000 bp.

The precision for 5000 bp input sequences highlights the severe impact of token limitations across nearly all models. Most models are entirely absent from the chart, having failed to return valid outputs due to exceeding their maximum input capacity.

`Agro-nucleotide-transformer-1b` and `nucleotide-transformer-v2-250m-multi-species` exhibit relatively modest precision scores hovering around 0.50-0.52. The `agro-nucleotide-transformer-1b` model exhibits a flat precision trend as a result of reaching its maximum input token capacity, while the `nucleotide-transformer-v2-250m-multi-species` does not reach its maximum limitations maintaining its precision nearly constant for all coverages.

Additionally, `gena-lm-bigbird-base-t2t` stands out with exceptional and consistently high precision, remaining above 0.92 across all coverage levels. This highlights its strong

capability to manage long-range dependencies and large input contexts without performance loss. Notably, it is the only model whose precision clearly improves with increasing input length, demonstrating its ability to effectively leverage extended contextual information. A more modest but still noticeable improvement is also observed in the `nucleotide-transformer-v2-250m-multi-species` model, suggesting that it too can benefit from longer context windows.

3.3.3 Recall

Recall measures a model's ability to correctly identify all relevant elements, in this case, the correct tokens from the target sequence. High recall indicates that the model is sensitive to the actual content to be predicted, minimizing false negatives.

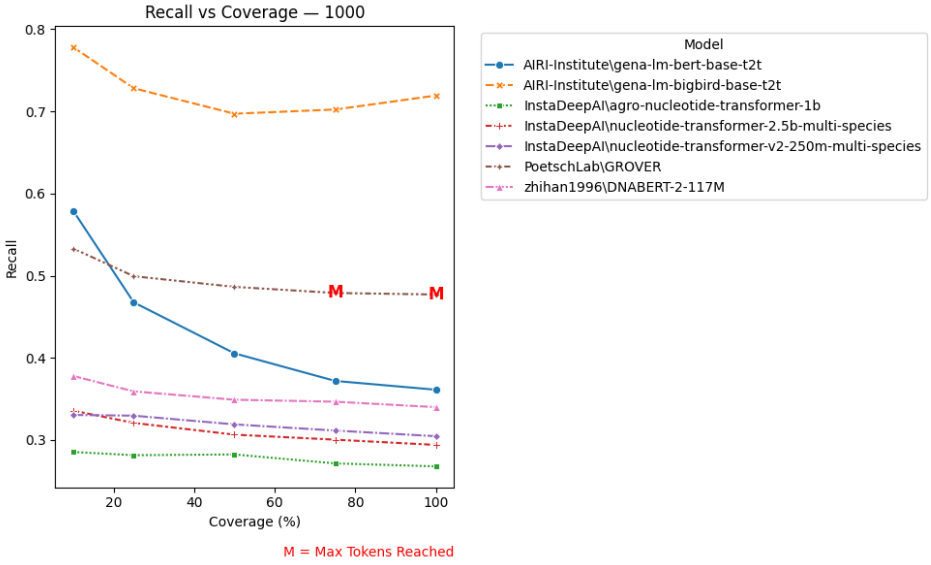

Figure 3.7: Recall vs Coverage with a context length of 1000 bp.

For 1000 bp sequences, at 10% coverage, `gena-lm-bert-base-t2t` starts off with a high recall, exceeding 0.58, indicating a strong ability to identify relevant tokens early in the prediction task. However, as coverage increases, its recall diminishes steadily, ending around 0.36 at 100%. This sharp decline suggests that the model becomes less sensitive to correct predictions as the length of the predicted sequence grows.

GROVER also begins with strong recall values just below 0.54, and while it too declines with increased coverage, its drop is more gradual and stabilizes at approximately 0.48. However, GROVER reaches its maximum token capacity at 75% coverage, limiting further recall improvement beyond this coverage.

Models such as `nucleotide-transformer-v2-250m-multi-species` and `nucleotide-transformer-2.5b-multi-species` show more modest recall values throughout. Their curves are relatively flat, with scores hovering around 0.30 to 0.33. This flatness suggests that these models do not significantly improve, or degrade, in recall performance across different coverage levels, possibly due to less effective contextual learning in shorter windows.

The DNABERT-2 model performs slightly better than the nucleotide-transformers, but still falls well behind the leading models, maintaining recall in the 0.34-0.38 range with a

slight downward slope.

`Agro-nucleotide-transformer-1b` ranks consistently as the lowest performer in recall across all coverage points, with values dipping below 0.28 and showing little sensitivity to increasing sequence length.

`Gena-lm-bigbird-base-t2t` clearly leads in recall performance, starting near 0.78 at 10% coverage and maintaining values above 0.70 across all levels. This reflects its strong ability to recover relevant nucleotides consistently, even as more of the sequence is generated. Unlike other models, its recall remains stable and high, highlighting its robustness in retaining correct predictions over longer spans.

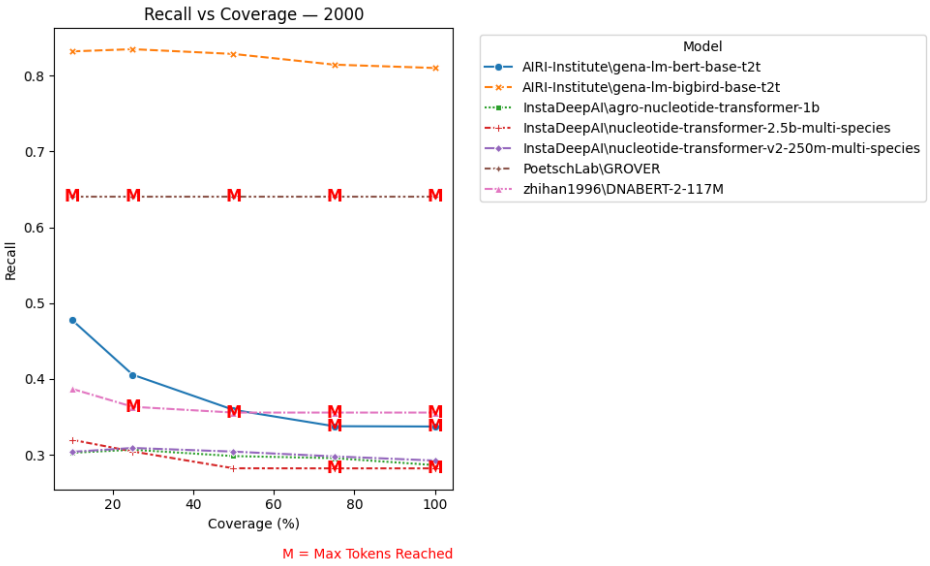

Figure 3.8: Recall vs Coverage with a context length of 2000 bp.

For 2000 bp sequences, the `gena-lm-bert-base-t2t` model begins with relatively strong recall, approaching 0.48 at 10% coverage. However, its performance diminishes consistently as coverage increases, dropping to around 0.34 at full coverage and reaching its input size limit at 75% coverage. This steady decline suggests that while the model is effective at recovering early segments of the target, its ability to recall relevant tokens decreases as the output sequence grows.

The GROVER model maintains a stable recall value just under 0.64 across all coverage thresholds. This consistency is a byproduct of hitting the model's token capacity at all levels of coverage. Its architectural limits prevent it from processing the full input.

Similarly, DNABERT exhibits a relatively flat recall curve, starting at about 0.39 and remaining near 0.35 beyond 25% coverage. This is a direct result of the model reaching its maximum token length early (at 25% coverage). Once this threshold is hit, further input has no effect on predictions, effectively freezing recall performance across the rest of the coverage range.

The nucleotide-transformer models (both 2.5b and v2-250m versions), along with the `agro-nucleotide-transformer-1b`, consistently show lower recall values, ranging narrowly from approximately 0.28 to 0.31. Their flat trends across coverage levels reflect limited capacity to recover relevant tokens, and no substantial benefit from additional context.

`Gena-lm-bigbird-base-t2t` maintains its lead with a high and steady recall, consistently above 0.81 across all coverage levels. This confirms its robust capacity to recover correct outputs across extended contexts, with no degradation despite the longer input. Its performance is not affected by input length or model constraints, indicating excellent scalability.

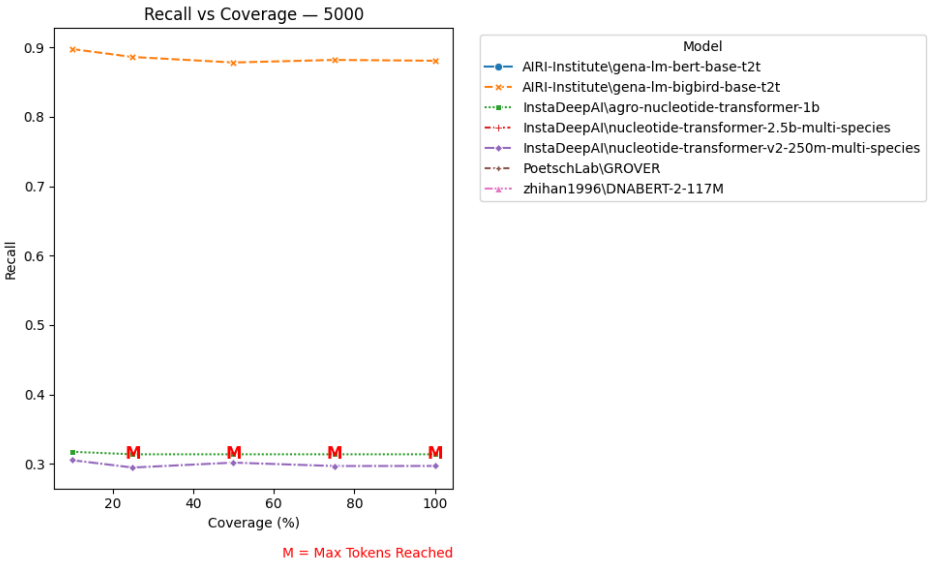

Figure 3.9: Recall vs Coverage with a context length of 5000 bp.

At 5000 bp, models such as GROVER, `gena-lm-bert-base-t2t`, and DNABERT-2 are absent from the recall curves entirely, as they are unable to process input sequences of this length, because their architectures do not support 5000-token contexts. Among the few models that do appear, both the `agro-nucleotide-transformer-1b` and the `nucleotide-transformer-v2-250m-multi-species` exhibit very modest and nearly flat recall curves. The agro model slightly leads with a recall value of around 0.317 at low coverage, but then it hits the token limit early (at 25%). The v2-250m model maintains values close to 0.3 throughout, with only minimal fluctuation. Neither model shows significant gains from increased coverage, again a symptom of architectural constraints or insufficient effective sequence processing depth.

In contrast, `gena-lm-bigbird-base-t2t` model once again stands out, maintaining exceptionally high and stable recall values, consistently around 0.88-0.90 across all coverage levels. This reaffirms its superior capacity to manage long-range dependencies and extract relevant tokens from extensive contexts, without degradation. Notably, it's the only model that continues to deliver such performance at this scale, and it does so without reaching token limitations.

3.3.4 F1 Score

The F1 Score results obtained across different models, sequence lengths (1000 bp, 2000 bp, and 5000 bp), and coverage levels (10%, 25%, 50%, 75%, and 100%) provide a balanced view of model performance by integrating both precision and recall. This metric is particularly informative for evaluating sequence prediction quality under varying input conditions.

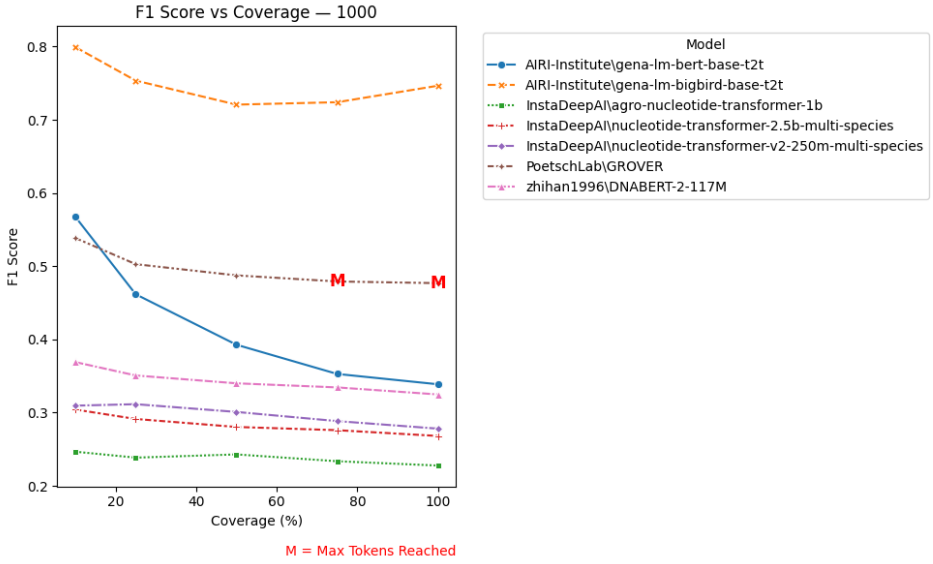

Figure 3.10: F1 Score vs Coverage with a context length of 1000 bp.

For the 1000 bp context, `gena-lm-bert-base-t2t` achieves a good F1 score at 10% coverage, reaching around 0.57, but this advantage erodes rapidly, falling to approximately 0.34 at full coverage.

GROVER, while starting slightly lower (~ 0.54), maintains its performance more gracefully, with a less steep decline and final scores that nearly match those of `gena-lm-bert-base-t2t` at the higher coverage levels. However, it reaches its token limit around 75% and 100% coverage.

Meanwhile, the remaining models, including DNABERT-2, `agro-nucleotide-transformer-1b`, and the two nucleotide-transformer models, display considerably lower F1 scores overall, with flatter curves and minimal responsiveness to increased coverage. Notably, DNABERT-2 consistently hovers in the 0.35 range and shows a steady but shallow downward trajectory, reaffirming the limited benefit from additional context.

`Gena-lm-bigbird-base-t2t` clearly dominates in terms of F1 Score, starting near 0.80 at 10% coverage and maintaining strong performance across all coverage levels with only a slight dip, remaining well above 0.70 throughout. This indicates a remarkable balance between precision and recall.

When evaluating the 2000 bp context, the `gena-lm-bigbird-base-t2t` model continues to dominate in terms of F1 Score, maintaining values above 0.85 across all coverage levels.

`Gena-lm-bert-base-t2t` again starts with a strong lead, posting an F1 score near 0.47 at 10% coverage. However, its score steadily declines as more of the target sequence is predicted, eventually leveling off around 0.33 at full coverage. Notably, unlike in the 1000 bp condition, `gena-lm-bert-base-t2t` begins to approach its token limit at higher coverage percentages.

GROVER maintains the highest overall F1 score across the entire coverage range. Yet, every data point is marked by a truncation flag, indicating that the model consistently hit its maximum token capacity.

Other models, including DNABERT-2 and the various nucleotide-transformer models vari-

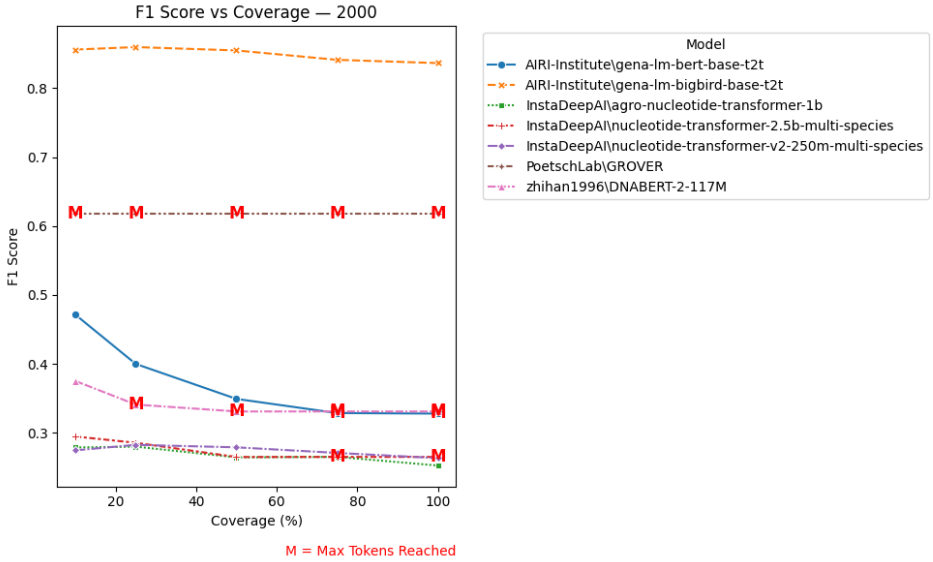

Figure 3.11: F1 Score vs Coverage with a context length of 2000 bp.

ants, remain notably behind. These models show relatively flat F1 curves that fluctuate minimally, suggesting they are less sensitive to input length but also less capable of leveraging additional context effectively. DNABERT-2, in particular, exhibits a nearly static F1 score from 25% coverage onward, a consequence of reaching the maximum token limit and being unable to incorporate further sequence context.

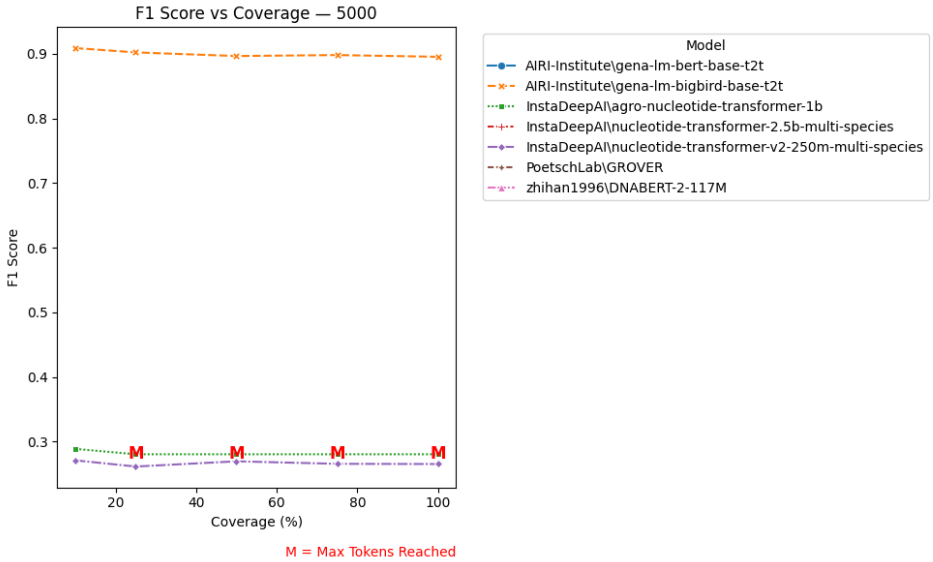

Figure 3.12: F1 Score vs Coverage with a context length of 5000 bp.

At the 5000 bp context length,the limitations imposed by model token capacities become even more evident. Most models are entirely absent from the curve, having reached their maximum token limits before being able to perform any meaningful predictions at this sequence length. `Agro-nucleotide-transformer-1b` and `nucleotide-transformer-v2-250m-multi-species` manage to report F1 scores across all coverage points.

However, their performance is modest and notably flat. The agro model achieves the highest F1 score in this set, around 0.288 at low coverage, but shows no measurable gain as cov-

erage increases, due to token limitations. Similarly, the `nucleotide-transformer-v2-250m-multi-species` model fluctuates slightly around the 0.265–0.270 range but offers no sign of improvement.

The `gena-lm-bigbird-base-t2t` model once again demonstrates outstanding performance, maintaining an F1 score consistently around 0.90 across all coverage levels. This not only confirms its robustness for long-context predictions but also highlights its ability to fully leverage extended input sequences without degradation.

3.3.5 Disk Usage

Disk usage was monitored during model inference to evaluate resource efficiency across different sequence lengths. The measurements reflect the total additional disk space used during the processing of each input file.

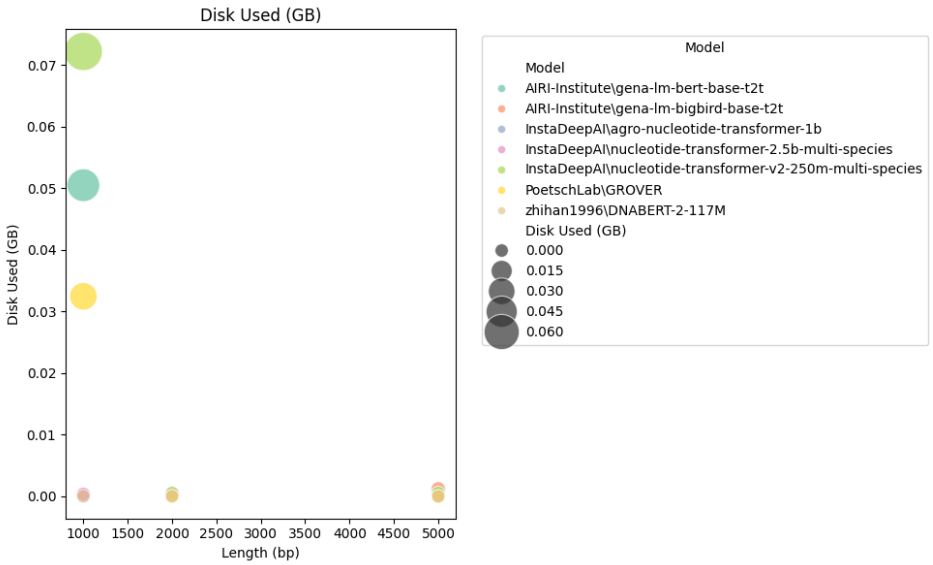

Figure 3.13: Disk Used vs Context Length.

The most striking observation is that disk consumption is concentrated almost exclusively at the shortest input length, 1000 base pairs. At this length, models like `nucleotide-transformer-v2-250m-multi-species` and `gena-lm-bert-base-t2t` consume noticeably more disk space, with the first one using over 0.07 GB, the seconnd one around 0.05 GB and GROVER over 0.03 GB. Other models, such as DNABERT-2, use considerably less, with values below 0.01 GB.

Interestingly, as the input length increases to 2000 bp and 5000 bp, disk usage drops sharply across all models, reaching near-zero levels. This behavior is largely attributed to input size limitations, which prevent models from processing longer sequences fully. As a result, the computational workload, and consequently the amount of intermediate data written to disk, remains minimal, since the models are unable to proceed with complete inference beyond their token constraints.

3.3.6 RAM Usage

RAM consumption was tracked during model execution to assess memory efficiency across different input sequence lengths. These measurements reflect the additional memory used

during inference for each input file.

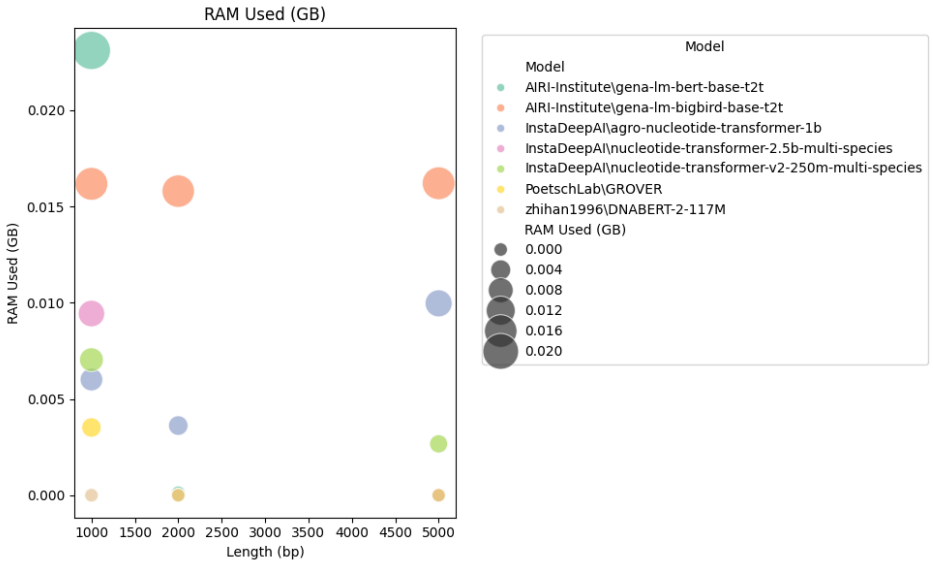

Figure 3.14: Ram Used vs Context Length.

The RAM usage chart reveals that memory consumption is highest for models when processing 1000 bp input sequences, particularly for `gena-lm-bert-base-t2t`, which peaks at around 0.022 GB and `gena-lm-bigbird-base-t2t` at 0.016 GB. Other models, such as `nucleotide-transformer-2.5b-multi-species`, `nucleotide-transformer-v2-250m-multi-species` and `agro-nucleotide-transformer-1b`, also show moderate memory usage at this sequence length.

Interestingly, as the input length increases to 2000 bp and 5000 bp, RAM usage drops significantly across all models except for `gena-lm-bigbird-base-t2t`, likely because this model never encounters token limitations and continues processing full-length inputs consistently. When models reach their maximum input capacity, they often process only a fraction of the sequence, leading to reduced computational load and lower memory demands. Overall, the trend highlights how memory usage is tightly linked to the portion of the input that the models are actually able to handle.

3.3.7 File Processing Time

The time required to process each input file was recorded in order to evaluate the computational efficiency of the models. This metric captures the full duration of inference, including model loading, tokenization, and sequential prediction.

The time chart provides a clear picture of the computational efficiency of each model as sequence length increases. Notably, `nucleotide-transformer-v2-250m-multi-species` and `gena-lm-bigbird-base-t2t` exhibit the highest processing times, particularly at 5000 bp, reaching nearly 1200 seconds per file, an indication of their heavy computational overhead. In contrast, models like GROVER and DNABERT-2 remain impressively fast, especially DNABERT-2, which stays under 5 seconds even for the longest input. The `gena-lm-bert-base-t2t` model also shows relatively moderate processing times, increasing slightly with sequence length but remaining within practical bounds.

These trends suggest that while some models may offer strong performance metrics, their

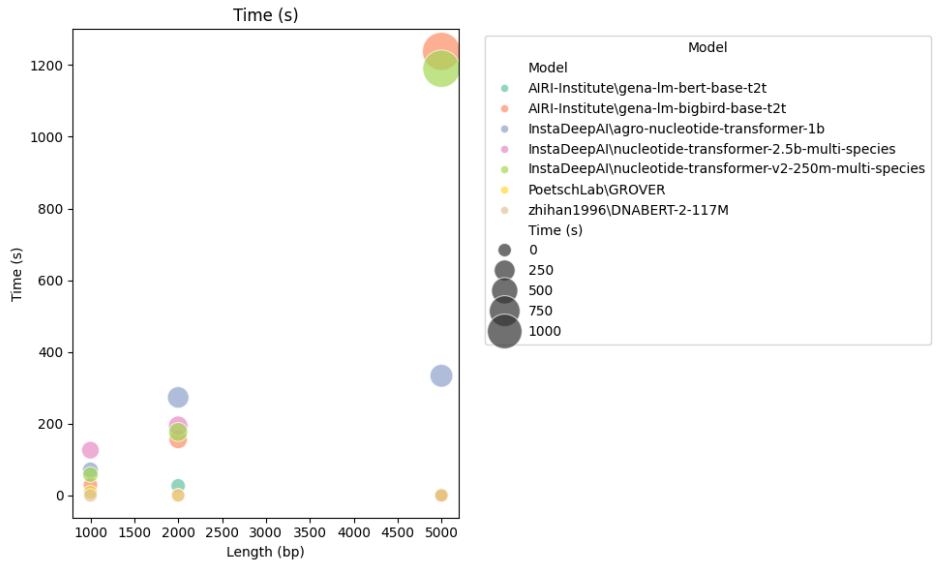

Figure 3.15: File Processing Time vs Context Length.

inference times can vary significantly, potentially limiting their scalability. Time efficiency is crucial for real-world deployment, especially when processing large datasets or performing batch inference, making lightweight models like DNABERT-2 and GROVER particularly attractive in time-sensitive scenarios.

3.3.8 Levenshtein Distance

The Levenshtein Distance chart offers a comprehensive view of how well each model reconstructs target sequences under varying input lengths, while also revealing the impact of token truncation. As the sequence length increases from 1000 to 5000 base pairs, the average Levenshtein distances tend to rise across nearly all models, indicating growing difficulty in maintaining accurate predictions over longer contexts. However, the extent of this deterioration varies significantly depending on the model and whether the predictions were truncated due to reaching the maximum token limit.

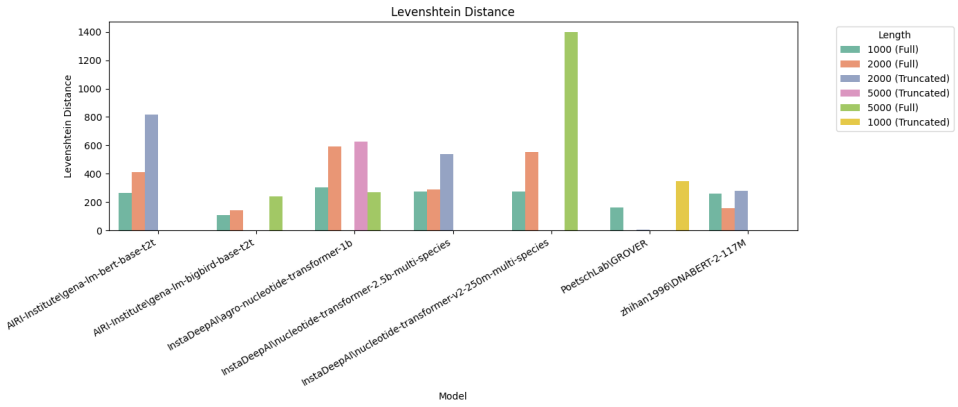

Figure 3.16: Levenshtein Distance vs Context Length.

Among the evaluated models, `gena-lm-bigbird-base-t2t` clearly stands out as the one achieving the lowest Levenshtein distances across all conditions where it produces outputs, including full-length predictions at 1000 bp 2000 bp and 500 bp. This superior

performance highlights its strong ability to generate sequences that closely resemble the reference.

GROVER and DNABERT-2 also show low Levenshtein distances, particularly in the 1000 bp and 2000 bp full-length conditions. This indicates their strong ability to generate sequences that closely match the target when operating within their token limits. However, beyond these input lengths, their performance data is missing due to truncation caused by maximum token capacity constraints. As a result, while both models are effective for shorter input contexts, they are not suitable for handling longer sequences.

In contrast, `gena-lm-bert-base-t2t` and `nucleotide-transformer-2.5b-multi-species` perform well on the 1000 bp input, but show a marked increase in Levenshtein distance for the 2000 bp truncated predictions. This sharp rise implies that hitting the token limit leads to incomplete or inaccurate outputs, undermining the overall fidelity of the sequence.

`Agro-nucleotide-transformer-1b` and `nucleotide-transformer-v2-250m-multi-species` exhibit moderate performance on shorter input contexts, but their Levenshtein distances increase notably as the input length grows. This steady rise suggests challenges in scaling effectively to longer sequences or generating accurate predictions when only partial outputs are feasible. Nonetheless, these two models stand out as the only ones capable of producing results for 5000 bp inputs, besides `gena-lm-bigbird-base-t2t`, indicating a stronger suitability for handling extended contexts compared to the rest.

3.4 Model Selection for Gap Prediction

Based on the comprehensive benchmarking results presented in the previous sections, we have selected **gena-lm-bigbird-base-t2t** as the most suitable model for gap prediction in genomic sequences.

This decision is supported by the following key findings:

- **Top-60 Accuracy:** gena-lm-bigbird-base-t2t consistently achieved the highest Top-60 Accuracy across all evaluated input lengths (1000 bp, 2000 bp, and 5000 bp) and coverage levels. Its performance remained stable and robust even as sequence coverage increased, demonstrating superior token ranking and contextual understanding.
- **Precision, Recall, and F1 Score:** This model outperformed all others across standard classification metrics. It maintained exceptional precision and recall throughout, leading to the highest F1 scores among the compared architectures. This indicates a strong balance between correct token selection and sequence completeness.
- **Scalability:** Unlike many models that failed or degraded when reaching their token limits, gena-lm-bigbird-base-t2t processed sequences of up to 5000 bp without truncation. This demonstrates its scalability and effectiveness for long-range DNA sequence modeling.
- **Computational efficiency:** Despite its large capacity, the model exhibited acceptable RAM usage and disk consumption across input sizes. While inference times were higher than for lighter models, this trade-off was justified by the consistent improvement in prediction accuracy.

- **Levenshtein distance:** The model achieved the lowest Levenshtein distances across all conditions, confirming its ability to reconstruct target sequences with high fidelity and minimal deviation from the ground truth.

Given its superior performance across both predictive and operational metrics, `gena-lm-bigbird-base-t2t` has been selected as the final model for downstream gap prediction tasks. Its robustness, accuracy, and ability to handle long contexts make it an optimal candidate for practical genomic sequence completion. Importantly, it was the model that benefited the most from increasing context length, using the additional information to improve output predictions. Unlike other models, its performance did not degrade as the target size grew, highlighting its scalability and effectiveness in long-range sequence modeling.

4. Gap Prediction Pipeline

In this chapter, we describe the development of a custom pipeline for predicting sequence gaps using the `gena-lm-bigbird-base-t2t` model, selected as the most robust and accurate architecture based on the benchmarking results discussed in the previous chapter.

To ensure a realistic and challenging evaluation environment, we constructed a new test dataset that simulates real genomic gaps as observed in biological DNA sequences. These artificial gaps are designed to closely replicate the structural characteristics, sequence composition, and length distributions of missing regions typically found in genome assemblies and sequencing outputs.

The pipeline integrates the pre-trained language model into an automated framework that takes incomplete DNA sequences as input, processes them iteratively, and generates high-confidence gap predictions. Each component of the pipeline, from data preprocessing and context window generation, to sequence reconstruction and output formatting, has been tailored to support accurate and scalable inference on long genomic sequences.

The following sections detail the design decisions, implementation strategy, and data preparation steps involved in building this gap prediction pipeline.

4.1 Gap Simulation Dataset

To evaluate the practical gap-filling capabilities of the selected model (`gena-lm-bigbird-base-t2t`), we constructed a specialized dataset that emulates the fragmentation patterns typically observed in real-world draft genome assemblies. The goal was to simulate realistic conditions for genome completion tasks under biologically plausible constraints.

This dataset was synthetically generated from a collection of complete bacterial genomes, each ranging from 500 kilobase pairs (kbp) to 5 megabase pairs (Mbp) in length.

The fragmentation process was carried out using a custom Python script developed with the `Biopython` library. This tool enabled the manipulation and parsing of genomic sequences in a reproducible and biologically meaningful way.

4.1.1 Simulation Procedure

For each complete genome:

- The sequence was fragmented into 50 to 200 contigs.
- Contig lengths were sampled from a log-uniform distribution between 1 kbp and 200 kbp, reflecting empirical distributions observed in bacterial draft assemblies.

- Between each contig, a gap region of unknown sequence was inserted. These gaps were randomly sampled from a uniform distribution between 0.5 kbp and 5 kbp, simulating unresolved regions typically encountered in assembly pipelines.

Sequential Assembly Simulation

The genome was processed linearly from start to end. At each step:

1. A contig length was drawn from the log-uniform distribution.
2. If sufficient sequence remained, a segment of that length was extracted and stored as a contig.
3. A gap of sampled length was added to simulate missing sequence information.
4. The process was repeated until less than 1 kbp remained in the genome.

4.1.2 Output Format and Metadata

Each simulated genome consists of:

- **FASTA file:** Contains all generated contigs, with headers indicating:
  - The original genome accession.
  - Unique contig ID (e.g., `contig50`).
  - The length of the contig.
  - The genomic coordinates from which the contig was extracted.

  Example header:

  ```
  >NZ_CP087381.1_contig50 length=199901 original_pos=1593829-1793730
  ```

- **Gap metadata file (TSV):** Contains the gap information between contigs, including:
  - Unique gap ID (e.g., `gap50`).
  - Start and end positions in the original genome.
  - Gap length.

  Example row:

ID	start	end	length
NZ_CP087381.1_gap50	1793730	1797567	3837

  This means `gap50` directly follows `contig50`, starting where the contig ends.

4.1.3 Purpose and Applications

This dataset is specifically designed to support the evaluation of gap-filling models under conditions that closely resemble real-world genome sequencing scenarios. The simulated

contigs and gaps mirror the typical patterns of fragmentation commonly encountered during DNA sequencing, making the dataset highly relevant for assessing model performance on practical genome completion tasks. It is particularly useful for:

- Evaluating gap prediction performance in bacterial genome reconstruction.
- Simulating realistic fragmented assemblies to test models on gap completion tasks.
- Training models to better capture sequence continuity across biologically plausible genomic gaps.

By incorporating realistic contig lengths and gap distributions, and by leveraging `Biopython` for consistent and reproducible genome manipulation, this dataset provides a robust foundation for benchmarking sequence completion models in the context of genomics.

4.2 Gap Prediction Pipeline Design

GENA-LM is a DNA language model that uses BPE as its tokenization method, with a vocabulary of 32,000 tokens. Unlike models such as DNABERT, which rely on fixed-length k-mer tokenization (e.g., 3-mers or 6-mers), BPE allows the model to learn and represent frequent nucleotide subsequences as single tokens. This results in a more flexible and data-driven representation of DNA. In practice, a single BPE token in GENA-LM can represent anywhere from 3 to 6 nucleotides, depending on sequence frequency and context. On average, 1 token corresponds to approximately 6 base pairs.

We use the `gena-lm-bigbird-base-t2t` variant, which is trained with a MLM objective. Rather than generating sequences autoregressively, the model learns to predict missing parts of a sequence when specific positions are replaced with the special `[MASK]` token.

To adapt this model for the gap-filling task, where we aim to infer missing regions between contigs, we simulate gaps by inserting a block of `[MASK]` tokens between the two known flanking regions. Because the model operates at the token level and each token may represent multiple nucleotides, the number of `[MASK]` tokens must be estimated based on the approximate size of the gap in nucleotides. We opted to use a block-style masking strategy because our task involves reconstructing continuous missing regions.

For each `[MASK]` token in the gap, the model generates multiple possible predictions. In our experiments, we sample several alternative completions for each masked block and evaluate them based on alignment to the ground-truth target sequence. This multi-sampling approach helps account for the stochasticity and ambiguity inherent in biological sequences.

The diagram below illustrates the overall pipeline used for model-based gap filling.

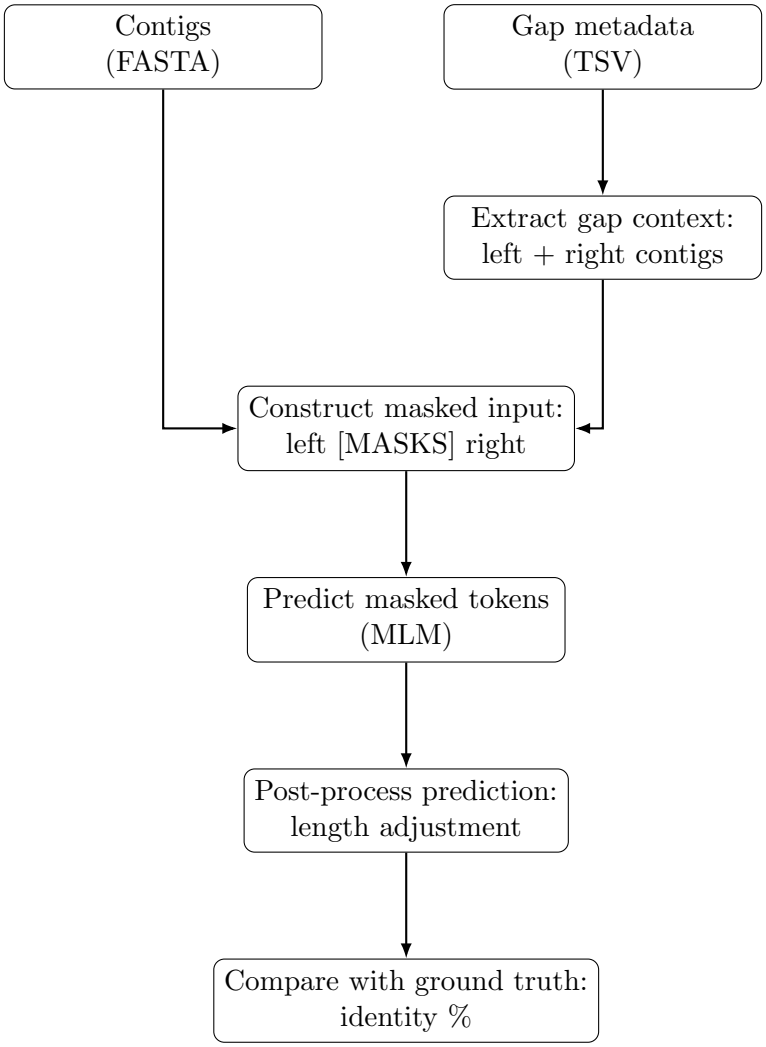

Figure 4.1: Gap-filling pipeline.

Pipeline Script: `gen_gap_filler.py`

A custom Python script was developed to automate this process. For a specific genome (e.g., `AP012051.1`), the script:

- Loads the simulated contigs and gap sequences.
- Selects the first three gaps for prediction.
- For each gap:
  - Finds the flanking contigs.
  - Inserts a block of `[MASK]` tokens between them.
  - Uses `gena-lm-bigbird-base-t2t` to predict the sequence of the gap.
  - Generates various alternative predictions.
- Stores all predictions and the ground truth sequences in a CSV file.

4.2.1 Tokenization Challenges and Dynamic Strategy

Initially, we considered inserting a number of `[MASK]` tokens equal to the number of missing nucleotides. However, this naive approach proved suboptimal for several reasons:

- BPE tokenization does not correspond 1:1 with nucleotide characters. A `[MASK]` token can represent varying numbers of nucleotides, and predicting a fixed number of masks often yields sequences shorter or longer than expected.
- Pre-tokenizing the actual gap to determine the number of tokens needed was also unreliable. Tokenization is context-dependent; the same gap sequence may yield a different token count depending on its flanking sequences.
- Forcing predictions to match the exact gap length through truncation or padding could distort biologically important features such as motifs or repeats.

To address these limitations, we implemented a dynamic and iterative mask prediction strategy, designed to align predicted output length with the actual gap size while preserving biological integrity. This strategy, reflected in the `predict_until_length` function, operates as follows:

1. An initial estimate of the number of `[MASK]` tokens is computed as the gap length divided by a scaling factor (typically 4), providing a starting point for the masked input.
2. The masked sequence (left flank + `[MASKS]` + right flank) is passed to the `gena-lm-bigbird-base-t2t` model. Predictions for masked tokens are sampled using a probability threshold, encouraging diverse but high-confidence outputs.
3. After each prediction, the model outputs a sequence of subword tokens. These tokens often include special characters used by the tokenizer (e.g., the underscore-like symbol, which marks word boundaries). We remove these artifacts and concatenate the tokens to reconstruct the actual nucleotide sequence predicted for the gap. Once cleaned, we measure the total number of nucleotides in the predicted sequence.
4. We then compare the predicted sequence length with the expected gap length. This gives us both the absolute difference (in number of nucleotides) and the relative difference (as a fraction of the expected gap). Based on how far off the prediction is, we adjust the number of `[MASK]` tokens for the next attempt: if the prediction is too short, we add more masks; if it's too long, we reduce them. The amount of adjustment is proportional to the size of the error, allowing for finer control as we approach the target length.
5. The process is repeated iteratively. The model dynamically adjusts the mask count with varying levels of aggressiveness depending on how far the current prediction deviates from the expected length. This adaptive control helps converge to a biologically plausible sequence of appropriate size.
6. The iteration halts once a prediction is within a small tolerance window (e.g., ± 3 nucleotides), or a maximum number of attempts to reduce the gap length and predicted gap length difference is reached (e.g., 10 attempts). The best-length match across all attempts is retained.

This adaptive strategy strikes a balance between structural fidelity and probabilistic sampling. By not hard-constraining the model and instead allowing it to refine predictions

over successive attempts, we harness its contextual understanding while aligning outputs with real genomic gaps. The approach is especially suited to complex or repetitive regions where fixed-length assumptions can be misleading.

4.2.2 Multiple Sampling

To account for the stochastic nature of masked language modeling, we generated multiple predictions per gap. The number of repetitions was defined by the `NUM_SAMPLES` parameter, set to 10 in our experiments. Each prediction was performed independently using the same flanking sequence context, allowing us to assess the consistency and variability of the model.

To ensure variation across predictions despite identical input contexts, we introduced controlled randomness into the token selection process.

Rather than selecting the most probable token at each `[MASK]` position (via `argmax`), we implemented a probabilistic sampling mechanism. Specifically, for each masked position:

- We computed the softmax probabilities of all tokens from the model's output logits.
- We filtered out low-probability candidates using a configurable threshold (e.g., $p \geq 0.01$).
- Among the remaining tokens, we performed multinomial sampling, selecting one token proportionally to its normalized probability.

To balance diversity and biological plausibility in the sampling process, we introduced a minimum probability threshold of 0.01. Only tokens with a model-assigned probability greater than or equal to 1% were considered viable candidates at each masked position.

This threshold was chosen empirically to strike a compromise between two competing goals:

- **Diversity**: lower thresholds increase variability across predictions, enabling exploration of multiple plausible gap sequences.
- **Quality**: higher thresholds ensure that only high-confidence tokens are selected, preventing random or biologically implausible insertions.

A threshold of 0.01 was found to be effective in generating meaningful variation across samples, while maintaining high alignment scores with the true sequences. It also reflects a commonly used range in probabilistic language modeling for sequence generation tasks.

This approach encourages the model to explore high-confidence alternatives without introducing random noise. As a result, each prediction can yield a distinct and plausible sequence, enabling the generation of multiple candidate gap fillings per context. This diversity is critical for downstream evaluation and for identifying sequences that align best with biological expectations.

4.2.3 Prediction Results

After each prediction, we computed the sequence identity between the predicted and true gap sequence using a global alignment algorithm (`pairwise2.align.globalxx`). The resulting alignment score was converted into a percentage of identity, defined as:

$$\text{Identity} = \frac{\text{Alignment Score}}{\max(\text{Length}_{\text{prediction}}, \text{Length}_{\text{true}})} \times 100$$

Each prediction, along with its associated metadata (gap ID, sample number, predicted sequence, reference sequence, gap length, and identity percentage), was stored in a structured list. Finally, the results were exported to a CSV file for further analysis, enabling easy visualization and post-processing of the model's performance.

Below, we present an example of the results obtained when attempting to fill three consecutive gaps in a simulated genome assembly. For each gap, the model generated multiple candidate sequences using only the preceding and following contigs as context. The table summarizes a subset of the outputs, including the predicted sequence, the real sequence, and the percentage identity between them based on global alignment.

As shown in the table, the identity percentages between the predicted sequences and the true gap sequences are considerably lower than in previous experiments, where the model was given a single continuous input sequence with a masked region to recover (i.e., context plus target). In the current setup, however, the model is presented with two disjoint contigs separated by an unknown gap, and must infer a plausible connecting sequence without any contiguous input across the masked region. Although the ground truth is still used exclusively for evaluation in both cases, this gap-filling configuration is inherently more difficult, as it offers less local context and requires the model to bridge two independent genomic fragments based on its internal knowledge of sequence continuity and structure.

Despite the use of probabilistic sampling and iterative adjustment to approximate the correct gap length, the predictions often deviate substantially from the true sequence. Nevertheless, this setup better simulates real-world scenarios in genome assembly, where the true sequence is unknown and the model is expected to infer biologically plausible fillers from incomplete data.

Gap ID	**#**	**Predicted**	**Real**	**Gap Length**	**Identity (%)**
AP012051.1_gap1	1	CTC...	TTT...	870	57.06
AP012051.1_gap1	2	GTC...	TTT...	870	56.95
AP012051.1_gap1	3	GTC...	TTT...	870	56.83
AP012051.1_gap1	4	GTC...	TTT...	870	57.01
AP012051.1_gap1	5	GTC...	TTT...	870	57.00
AP012051.1_gap1	6	CTC...	TTT...	870	56.67
AP012051.1_gap1	7	GTC...	TTT...	870	56.88
AP012051.1_gap1	8	GTC...	TTT...	870	57.04
AP012051.1_gap1	9	CTC...	TTT...	870	57.11
AP012051.1_gap1	10	GTC...	TTT...	870	56.93
AP012051.1_gap2	1	CCA...	TAC...	4867	54.22
AP012051.1_gap2	2	CAC...	TAC...	4867	54.22
AP012051.1_gap2	3	GAC...	TAC...	4867	54.26

AP012051.1_gap2	4	CAC...	TAC...	4867	54.20
AP012051.1_gap2	5	CAC...	TAC...	4867	54.30
AP012051.1_gap2	6	CAC...	TAC...	4867	54.24
AP012051.1_gap2	7	GAC...	TAC...	4867	54.18
AP012051.1_gap2	8	CCA...	TAC...	4867	54.22
AP012051.1_gap2	9	GAC...	TAC...	4867	54.24
AP012051.1_gap2	10	GAC...	TAC...	4867	54.18
AP012051.1_gap3	1	CAA...	GAG...	936	43.91
AP012051.1_gap3	2	GAA...	GAG...	936	44.02
AP012051.1_gap3	3	CAA...	GAG...	936	43.91
AP012051.1_gap3	4	CAA...	GAG...	936	43.91
AP012051.1_gap3	5	CAA...	GAG...	936	43.91
AP012051.1_gap3	6	GAA...	GAG...	936	44.02
AP012051.1_gap3	7	GGA...	GAG...	936	44.02
AP012051.1_gap3	8	CAA...	GAG...	936	43.91
AP012051.1_gap3	9	CAA...	GAG...	936	43.91
AP012051.1_gap3	10	CAA...	GAG...	936	43.91

Table 4.1: Example predictions for gaps in genome *AP012051.1*.

4.3 Integration of RAG into the Gap Prediction Pipeline

As observed in the previous section, the accuracy of the baseline model in predicting gap sequences, based solely on the flanking contigs, was relatively low, with identity percentages often below 60%. Despite iterative tuning and probabilistic sampling, the model struggled to generate biologically accurate gap completions when relying purely on its internal representations.

To address this limitation, we explored the integration of RAG into the pipeline. The motivation behind this approach is to enhance the model's predictions by providing it with additional external information retrieved from a database of known genomic sequences. The idea is that, by exposing the model to similar real sequences from other genomes or regions, it can make more informed and biologically plausible predictions for the missing gap content [45].

The RAG-augmented pipeline retrieves sequences that share local similarity with the flanking regions. These retrieved examples are inserted into the input prompt and serve as soft guides during the prediction process. This technique effectively extends the model's contextual window beyond what the architecture alone would permit, and introduces relevant, evolutionarily grounded patterns that may help the model resolve ambiguity in the missing region.

By comparing the results of the baseline and RAG-enhanced predictions, we aim to assess

whether incorporating retrieval improves identity scores and overall sequence coherence.

4.3.1 Species Identification and RAG Corpus Construction

To build the retrieval corpus for RAG-based gap prediction, we leveraged the contigs already present in our simulated dataset. Since these contigs originated from real reference genomes, we first aimed to identify the species associated with each contig in order to retrieve relevant genomic sequences for use as external context.

To do so, we developed a Python script (`identify_species_from_accessions.py`) that extracts accession numbers from all FASTA files in the dataset and queries the NCBI Nucleotide database to determine the corresponding species. The script performs the following steps for each unique accession:

1. The script scans all FASTA files in the dataset and extracts unique accession prefixes from the contig headers (e.g., `AP012051.1` → `AP012051`).
2. For each accession, it uses `Entrez.esearch` to retrieve the corresponding unique identifier (UID) from the NCBI Nucleotide database.
3. Using this UID, it fetches the GenBank record via `Entrez.efetch` and parses the `ORGANISM` field to extract the species name.
4. The results are stored in a CSV file (`contig_accessions_species.csv`), which maps each accession to its species of origin.

This mapping allows us to associate each contig with a taxonomically meaningful label. An excerpt of the resulting dataset is shown below:

Accession	**Species**
CP138982	*Winogradskyella sp. MIT101101*
CP129556	*Staphylococcus aureus*
NZ_CP027806	*Cyclonatronum proteinivorum*
NZ_CP003409	*Thermotoga sp. Cell2*
CP082927	*Candidatus Peribacteria bacterium*

Table 4.2: Accession numbers and their associated species.

This species-level annotation was then used to retrieve complete reference genomes from public databases (e.g., NCBI RefSeq and GenBank), ensuring that the RAG corpus would contain sequences relevant to the same organisms represented in the gap prediction dataset.

This species-aware construction of the retrieval corpus helps ensure that the additional context provided to the model during inference is biologically relevant and taxonomically coherent, thereby increasing the chances of generating accurate and realistic gap predictions.

Selection of Top Species

After identifying the species associated with each contig in our dataset, we proceeded to determine which species were most frequently represented. This frequency analysis allowed us to prioritize which reference genomes to include in the RAG corpus, under the assumption that genomes from the most common species would be more relevant for the prediction tasks.

Initially, we considered retrieving reference genomes for all species identified in the contig dataset. However, this approach proved to be impractical in terms of computational resources. Downloading and indexing the full genomic data for more than 100 unique species resulted in excessive disk space usage and memory consumption, and several genomes failed to process completely due to size, redundancy, or download errors.

To address this limitation, we restricted the retrieval step to the most common species in the dataset. By selecting only the top `N` species (e.g., the 20 most frequent ones), we significantly reduced the size of the RAG corpus while still preserving the taxonomic diversity most relevant to the input contigs.

To automate this process, we developed a script that reads the output of the species identification step (`contig_accessions_species.csv`) and performs the following operations:

1. Counts how many times each species appears in the dataset.
2. Selects the top `N` species (e.g., top 20) based on frequency.
3. Formats the species names into a sanitized Bash array suitable for command-line usage (removing special characters, converting to lowercase, etc.).
4. Automatically generates a Bash script (`download_genomes.sh`) that automates the genome retrieval.

An excerpt of the resulting species frequency table is shown below:

Species	**Count**
Staphylococcus aureus	3
Dehalococcoides mccartyi	3
Campylobacter lari	2
Akkermansia muciniphila	2
Fusobacterium animalis	2

Table 4.3: Most frequent species found in the contig dataset (Saved in `top_species.csv` dataset).

RAG Corpus Construction

The script `download_genomes.sh` performs the following tasks:

- Creates the directories `genomes_downloaded` and `rag_corpus` if they do not already exist.
- Iterates over the top list of species and for each:

  - Queries the NCBI Assembly database using `esearch` and `efetch` to find the latest complete genome.
  - Retrieves either the RefSeq or GenBank FTP path associated with that genome.
  - Searches for a `.fna.gz` file containing the genomic sequence.
  - Downloads the file and stores it in a species-specific subdirectory.
  - Copies the compressed `.fna.gz` file into the central `rag_corpus` directory.

- Finally, decompresses all `.fna.gz` files within `rag_corpus`, resulting in a set of plain FASTA files that serve as textual input for the RAG indexing process.

This procedure ensures that the RAG documents originate from genomes closely related to those observed in our simulated contigs, improving the biological relevance of the retrieval process.

4.3.2 Gap Prediction Pipeline Design using RAG

After constructing the RAG corpus from relevant genomic sequences (`.fna` files), we extended our original gap prediction pipeline by incorporating a RAG mechanism. The primary objective of this integration is to enrich the model's input with external genomic context that may not be present in the immediate flanking regions of the gap. By retrieving semantically similar sequences from a pre-indexed corpus using vector-based similarity search, we provide the model with additional, biologically relevant information to suppoort its predictions.

This RAG-enhanced approach is useful as local sequence context is insufficient to resolve the complex gaps with a high accurcay. The retrieved context serves as an auxiliary source of evidence, complementing the original input and allowing the model to better infer plausible completions for missing segments.

Below, we present a schematic diagram of the gap-filling pipeline enhanced with RAG. The system includes a retrieval component that queries a vector store of genomic embeddings and injects the most relevant results into the model's input, thereby enabling context-aware sequence reconstruction.

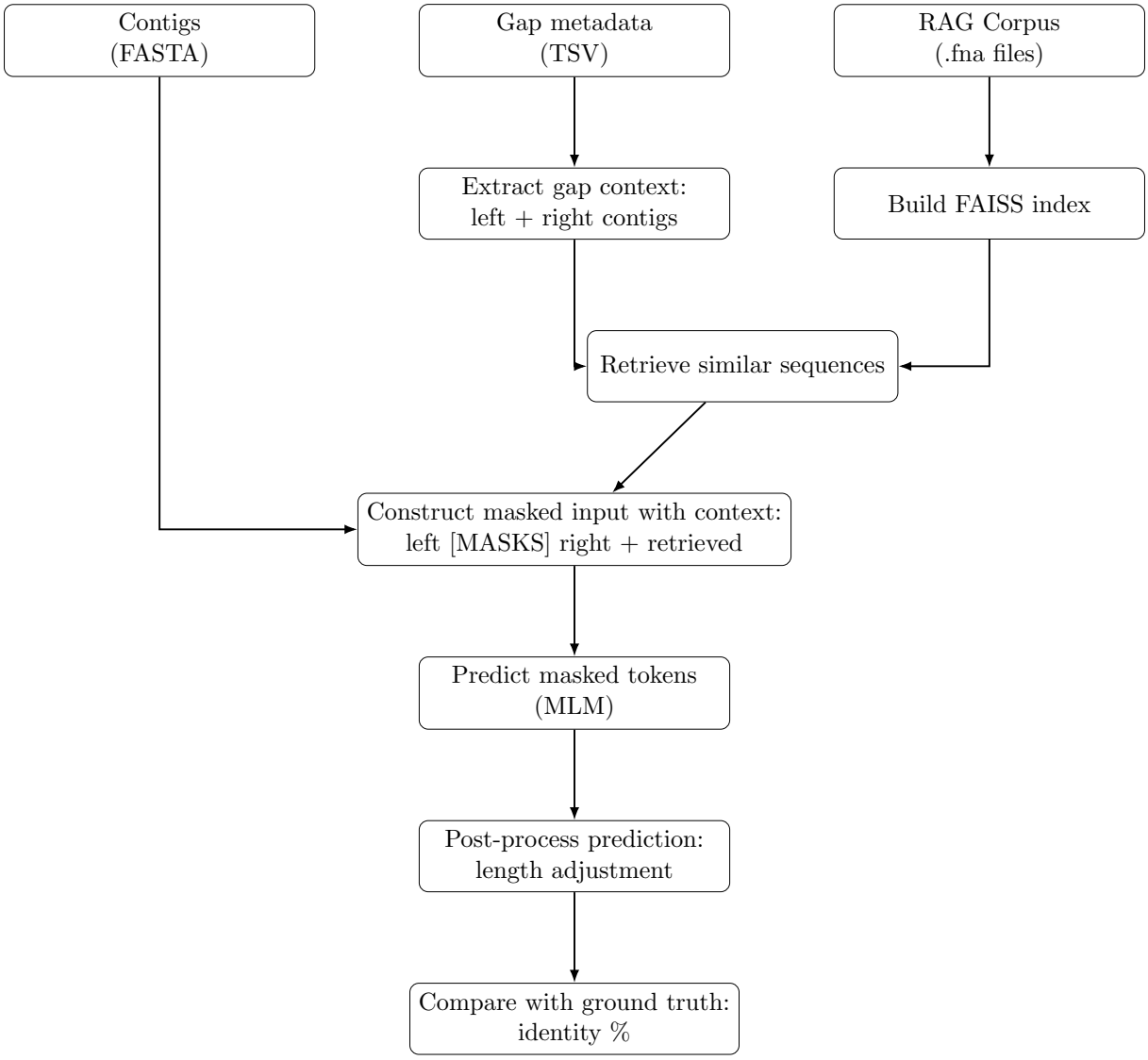

Figure 4.2: Gap-filling pipeline with RAG.

The RAG-enhanced pipeline follows several key steps:

1. **Index Construction:** we first build a dense vector index from the downloaded `.fna` files using the FAISS library. Each file is parsed to extract DNA sequences, which are wrapped as LangChain `Document` objects. These documents are then split into overlapping chunks of 1000 base pairs (with 200 bp overlap) using the `RecursiveCharacterTextSplitter` module. The resulting chunks are embedded using the `all-MiniLM-L6-v2` model from Sentence Transformers. The final FAISS index is persisted locally for reuse.

2. **RAG-based Query Retrieval:** for each gap to be filled, the query is formed by concatenating the left and right contig sequences that flank the gap. This query is used to retrieve $k = 2$ most similar chunks from the FAISS index. These retrieved DNA sequences serve as external genomic context.

3. **Contextual Input Construction:** the base input to the model follows the format `LEFT_SEQ [MASKS] RIGHT_SEQ`. In the RAG-enabled version, the retrieved context is appended as a comment-like section to form:

`LEFT_SEQ [MASKS] RIGHT_SEQ \\Context: <retrieved_chunk>`

4. **Mask-Based Prediction:** we use the `gena-lm-bigbird-base-t2t` model. The number of `[MASK]` tokens is dynamically adjusted across multiple iterations using a feedback loop based on the difference between the predicted and target gap lengths. Tokens are sampled proportionally from those surpassing a softmax threshold of 0.01, introducing controlled randomness for generating diverse predictions. The methoodlogy followed was explained with more detail in the previous section.
5. **Evaluation:** for each gap, we generate 10 predictions, both with and without RAG. Each predicted sequence is compared to the ground-truth gap using pairwise alignment, and the identity percentage is calculated. This allows us to quantitatively evaluate the impact of RAG on prediction accuracy.

The code responsible for this process is organized around two main functions:

- `build_faiss_index()`: Loads, chunks, embeds, and indexes the `.fna` sequences.
- `run_pipeline(use_rag=True)`: Executes the prediction process, optionally using the FAISS index for context retrieval.

The results from the RAG pipeline are saved as a CSV file. Below we can observe a table containing the results obtained for the first three gaps in a genome sequence.

Gap ID	#	Predicted	Real	Gap Length	Identity (%)
AP012051.1_gap1	1	GTC...	TTT...	870	55.98
AP012051.1_gap1	2	GGT...	TTT...	870	55.86
AP012051.1_gap1	3	GTC...	TTT...	870	56.32
AP012051.1_gap1	4	CTC...	TTT...	870	55.75
AP012051.1_gap1	5	CTC...	TTT...	870	55.75
AP012051.1_gap1	6	GTC...	TTT...	870	56.44
AP012051.1_gap1	7	GTC...	TTT...	870	55.75
AP012051.1_gap1	8	CTC...	TTT...	870	56.32
AP012051.1_gap1	9	GGT...	TTT...	870	55.75
AP012051.1_gap1	10	CTC...	TTT...	870	55.75
AP012051.1_gap2	1	CCT...	TAC...	4867	53.38
AP012051.1_gap2	2	CTC...	TAC...	4867	53.40
AP012051.1_gap2	3	GTC...	TAC...	4867	53.39
AP012051.1_gap2	4	GGT...	TAC...	4867	53.38
AP012051.1_gap2	5	GTC...	TAC...	4867	53.39
AP012051.1_gap2	6	GCT...	TAC...	4867	53.37
AP012051.1_gap2	7	CTC...	TAC...	4867	53.40
AP012051.1_gap2	8	GGT...	TAC...	4867	53.37
AP012051.1_gap2	9	GCT...	TAC...	4867	53.40
AP012051.1_gap2	10	GTC...	TAC...	4867	53.40

AP012051.1_gap3	1	GAA...	GAG...	936	44.02
AP012051.1_gap3	2	GAA...	GAG...	936	44.02
AP012051.1_gap3	3	GAA...	GAG...	936	44.02
AP012051.1_gap3	4	CAA...	GAG...	936	43.91
AP012051.1_gap3	5	GGA...	GAG...	936	44.02
AP012051.1_gap3	6	CAA...	GAG...	936	43.91
AP012051.1_gap3	7	GAA...	GAG...	936	44.02
AP012051.1_gap3	8	GGA...	GAG...	936	44.02
AP012051.1_gap3	9	CAA...	GAG...	936	43.91
AP012051.1_gap3	10	CAA...	GAG...	936	43.91

Table 4.4: Example predictions for gaps in genome *AP012051.1* using RAG.

The table above shows, for each gap, the identity percentages between the predicted sequences and the true genomic sequences. Based on these results, we now visualize whether incorporating retrieval-augmented context leads to meaningful improvements.

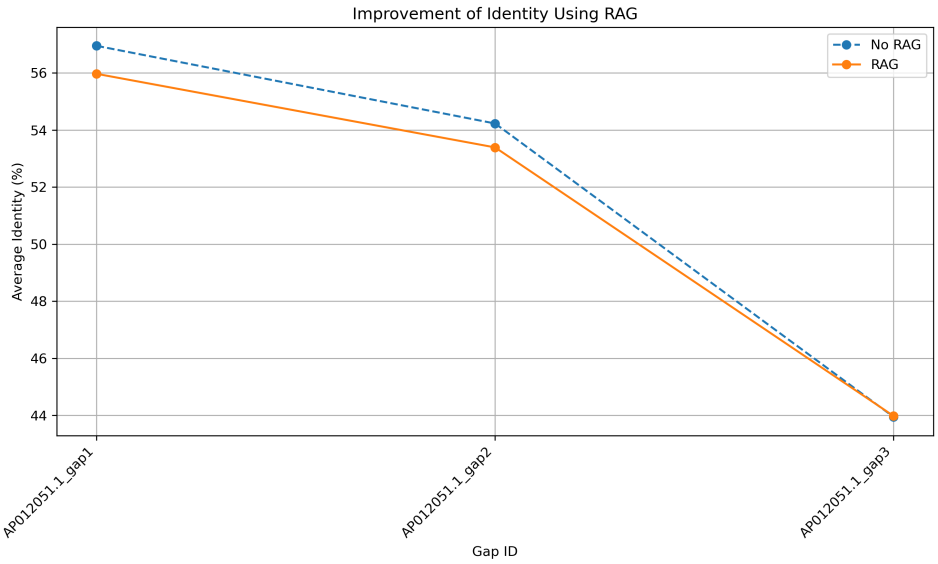

Figure 4.3: RAG vs no RAG identity comparison.

As shown in Figure 4.3, we compare the average identity percentages between predicted sequences and the ground truth for three gaps from genome *AP012051.1*, using both the standard MLM approach and the same method augmented with RAG.

Across all three gaps, we observe that the use of RAG does not consistently improve prediction performance. In fact, for every gap analyzed, the identity percentages obtained with RAG are slightly lower than those without it, although the differences are relatively small (typically under 2 percentage points). For example, in `gap1` and `gap2`, predictions without RAG achieved slightly better identity scores, while for `gap3`, both methods converged to a similar performance near 44%.

One likely reason for this lack of improvement is the nature of the underlying model. The selected model is MLM, which is inherently not designed to leverage long contextual prompts in the same way that decoder-only autoregressive models do [46]. Although we appended external genomic context to the input string, the model treats this added text as indistinguishable from the rest of the sequence and does not explicitly associate it with the masked region. Consequently, the retrieved context may have minimal or even negative influence on the model's ability to generate accurate gap predictions.

These results highlight a fundamental limitation in applying RAG to MLMs in genomic tasks. While the idea of enriching the input with similar sequences is appealing, a masked transformer cannot dynamically attend to external documents unless it is explicitly trained or fine-tuned for that behavior. To fully exploit the potential of retrieval-based augmentation, future work may explore the use of decoder-based models or encoder-decoder architectures that are better suited for conditional generation tasks and can more directly benefit from retrieved context.

As a result, this form of RAG integration proves suboptimal. To address this, we propose a different approach: incorporating retrieval via agent-based pipelines that use the retrieved context in a more structured and iterative manner. Agents can reason over retrieved sequences, invoke specialized models for different tasks, and potentially improve gap-filling accuracy through more dynamic use of external information.

In summary, although technically feasible, integrating RAG into this MLM pipeline had limited effect on prediction performance. This outcome underscores the need for alternative strategies, such as agent-based architectures or decoder-based models, to more effectively leverage retrieval in genomic sequence reconstruction.

5. Agent-based Gap Prediction Pipeline

Motivated by the limitations observed when applying RAG to MLM models, we explore an alternative approach to enhance the gap prediction task using agents. Agent-based pipelines allow for modular, step-by-step decision-making, which can include context retrieval, sequence formatting, iterative prediction, and quality evaluation, each as discrete actions controlled by the agent [47].

In the following section, we describe how agents can be integrated into the gap-filling pipeline to orchestrate retrieval, prediction, and evaluation more effectively. This method opens the door to more adaptable and intelligent workflows capable of leveraging both external knowledge and domain-specific heuristics.

5.1 Local vs. Cloud LLM Deployment

To support the agent-based architecture for gap prediction, we evaluated two complementary deployment strategies for the language model backend: local and cloud-based.

5.1.1 Cloud Deployment

Cloud deployment offers scalability, ease of integration, and access to the latest models without the need to manage infrastructure. It allows teams to quickly experiment with high-capacity LLMs while benefiting from automatic updates and vendor-provided optimizations [48]. However, these benefits come with notable drawbacks, including recurring usage costs, potential network latency, and data privacy concerns.

Cloud-based LLMs provide unmatched resource scalability and seamless deployment, but reduce control over infrastructure and can incur higher long-term costs [49]. Furthermore, AI workloads often face traditional cloud constraints, such as pricing unpredictability, limited GPU availability, and compliance issues, which motivate organizations to explore hybrid or edge strategies.

5.1.2 Local Deployment

Local deployment places the LLM entirely within on-premises infrastructure. This grants full control over hardware, ensures data privacy, and eliminates dependency on internet connectivity. Such configurations are particularly relevant for domains with strict data governance requirements, such as genomics.

The trade-offs include higher upfront investment, the need for specialized hardware, and manual handling of updates and model optimization [50]. While access to certain proprietary or cutting-edge models may be limited, local deployments ensure operational independence and predictable costs over time.

5.1.3 Hybrid and Edge-Based Approaches

Hybrid strategies combine both paradigms by delegating lightweight or sensitive tasks to local models while outsourcing computationally intensive workloads to the cloud. This approach aims to optimize latency, reliability, and cost while maintaining compliance with privacy regulations.

Recent work on Enhanced Hybrid Inference Techniques proposes a framework for scalable personalization of LLMs that integrates on-device processing with dynamic cloud resource allocation. This approach reduces latency, minimizes reliance on continuous internet connectivity, and enables real-time personalization, while offloading complex computations to the cloud when necessary to maintain high performance [51].

5.2 Exploring Local LLM Deployment with LLaMA.cpp

To integrate RAG with an agent-based architecture for genomic gap prediction, we opted to use `LangChain` as the orchestration framework. LangChain is a modular framework designed to facilitate the development of applications powered by LLMs, with built-in tools for retrieval, memory, prompt templating, agent planning, and multi-step reasoning [52].

In our context, LangChain enabled the construction of a pipeline in which a LLM could be repeatedly queried and guided through a multi-step agent to perform DNA gap inference. The agent combines sequence-based reasoning with structured metadata from contigs and surrounding genomic context, retrieved via RAG. This architecture allows the model to make informed decisions about gap completion by iteratively accessing both internal model knowledge and external genomic evidence.

While cloud-based APIs such as those offered by OpenAI or Anthropic provide access to state-of-the-art language models, we chose to begin our experimentation using a locally hosted, open-source model. This decision aligned with our goals of maintaining full control over the model's behavior, ensuring reproducibility, and supporting low-latency interactions within the gap prediction pipeline. Additionally, deploying the model locally allowed us to operate within the project's computational and budgetary constraints, while enabling flexible integration with tools such as LangChain and vector-based retrieval components.

As a first step toward integrating local agents into the gap prediction pipeline, we experimented with **Cortex**, a lightweight framework for running large language models locally. Cortex offers a convenient OpenAI-compatible REST API and supports interactive usage with models such as LLaMA via terminal or Python applications.

However, during development, Cortex proved unstable and incompatible with our use case. Additionally, the project transitioned into the more actively developed `llama.cpp`

repository.

`Llama.cpp` is a C/C++ framework designed to run LLaMA models efficiently on a wide range of devices with minimal hardware requirements. Its low overhead, ease of customization, and support for CPU-only inference made it a practical option for deploying models in flexible environments. The availability of Python bindings through the `llama-cpp-python` package also facilitated seamless integration with our existing tools, such as LangChain [53]. For these reasons, we selected `llama.cpp` as our backend for local agent-based inference.

5.2.1 Initial attempts with Cortex.

We initially explored the use of `Cortex`. Cortex offered a simplified user experience, allowing easy interaction through REST endpoints or Python wrappers using the `openai` client. However, shortly after beginning our experiments, the Cortex project was marked as *read-only*, and its development officially transitioned to the more active and robust `llama.cpp` repository. This shift also introduced architectural changes, making Cortex no longer compatible with recent model formats like GGUF.

Consequently, our attempts to integrate Cortex into the agent pipeline were no longer viable. We therefore decided to adopt `llama.cpp`, which continues to be actively maintained, supports the latest quantized models, and provides a flexible and efficient backend for local inference with LLaMA models.

5.2.2 Deploying LLaMA 2 locally using `llama.cpp`.

The `llama.cpp` project offers a C++-based backend to run LLaMA models efficiently on consumer hardware. It supports quantized GGUF models which drastically reduce memory consumption without significantly sacrificing accuracy. We selected the `LLaMA 2 7B Chat` model, quantized with `Q5_K_M`, and requiring 7 GB of RAM. This setup ensured compatibility with most local development environments.

5.2.3 Installation and Setup Steps

1. **Clone and compile the `llama.cpp` repository:**

   ```
   git clone https://github.com/ggml-org/llama.cpp
   cd llama.cpp
   make server
   ```

   This step produces a binary `server` capable of serving the model through a REST API.

2. **Download the quantized model (GGUF):**

   ```
   mkdir model && cd model
   wget -O llama-2-7b-model.gguf \
     "https://huggingface.co/TheBloke/Llama-2-7B-Chat-GGUF/resolve/main/

   llama-2-7b-chat.Q5_K_M.gguf"
   ```

The selected quantization `Q5_K_M` balances performance and resource usage, making it feasible to run on standard laptops.

3. **Create a Docker image for deployment:** since dependency management and compilation may vary by OS, we built a containerized environment. A minimal Dockerfile was written as `Dockerfile.server`:

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
    curl \
    git \
    wget \
    python3 \
    python3-pip \
    libopenblas-dev \
    libcurl4-openssl-dev \
    ninja-build \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY . /app

RUN cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && \
    cmake --build build --config Release --target llama-server -j

EXPOSE 8080

CMD ["./build/bin/llama-server", "-m", "/models/llama-2-7b
model.gguf",

"--port", "8080"]
```

The image was built using:

```
docker build -t llama-server -f Dockerfile.server .
```

4. **Run the server with Docker:**

```
docker run --rm -it -p 3928:8080 \
  -v $(pwd)/../models:/models \
  llama-server \
  ./server -m /models/llama-2-7b-model.gguf --port 8080
```

This command binds port 8080 inside the container to 3928 on the host and exposes the model through an OpenAI-compatible REST endpoint.

5. **Test server with `curl`:**

```
curl http://localhost:3928/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "llama-2-7b-model",
    "messages": [{"role": "user", "content": "What is DNA?"}]
  }'
```

A successful response from the server confirms correct setup and model loading.

6. **Integration with LangChain:** a custom class was created to wrap the local model as a LangChain-compatible LLM. It uses `requests` to call the REST API, with the endpoint configured as:

`http://localhost:3928/v1/chat/completions`

This allowed seamless use of the model within the LangChain agent framework.

5.2.4 Final Working Setup

After several attempts with both custom Dockerfiles and manual compilation of `llama.cpp`, we finally achieved a functional local deployment using the official pre-built Docker image provided by the GGML project. The deployment was carried out from a Windows environment using PowerShell with the following command:

```
docker run -v "C:\Users\...\gap-filler-agents\models:/models" \
-p 8000:8000 ghcr.io/ggml-org/llama.cpp:server \
-m /models/llama-2-7b-model.gguf --port 8000 --host 0.0.0.0 -n 512
```

This command mounts the local directory containing the GGUF model into the Docker container and exposes port `8000` for interaction. The model `llama-2-7b-chat.Q5_K_M.gguf` was previously downloaded from Hugging Face, as in earlier steps.

Once the server was running, it could be queried via the standard OpenAI-compatible endpoint:

```
curl http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [
      {"role": "user", "content": "What is DNA?"}
    ]
  }'
```

A successful response confirmed that the server was correctly exposing the model through a REST API and capable of processing basic prompts.

After deploying the model successfully using `llama.cpp`, we verified its availability through the exposed REST API endpoint on `localhost:8000`. In addition to supporting API-

based interaction, the server also offers a simple web interface where users can interact with the model directly. This interface allows for rapid prototyping and manual inspection of model behavior during debugging.

Below we present a screenshot of the running server's web UI.

Figure 5.1: Web interface of the local `llama.cpp` server running the LLaMA 2 7B model.

5.2.5 Outcome and Limitations

While deploying LLaMA 2 locally via llama.cpp was ultimately successful, and the server interface functioned as expected, the model failed to meet the requirements for our genomic gap-filling pipeline. Although the REST API was responsive and allowed direct interaction through curl, integration into our agent-based pipeline was severely constrained. A key limitation was that the model did not support tool binding in LangChain, a functionality essential for our workflow. Tool binding allows an agent to dynamically connect the language model to external tools, such as the RAG component and the gap filler, so that predictions can be combined with domain-specific operations. Without this capability, the model could not be orchestrated within our pipeline in a modular way, preventing the use of the iterative, tool-augmented strategy that the agent architecture relies on.

Beyond the lack of tool integration, the model also struggled to process long or complex DNA prompts, and tokenization or instruction formatting often appeared misaligned with expectations.

This highlighted a fundamental limitation: despite being lightweight and accessible, the deployed model lacked both the necessary instruction tuning and the architectural support for tool-enabled agent workflows. Furthermore, larger and more powerful alternatives (e.g., GPT-4, Claude 3) were not viable due to project resource constraints and the need for full local control.

For these reasons, we concluded that a new approach was required. We decided to search for an alternative architecture or framework to support our agent-based pipeline, including the use of function-calling agents and more specialized models in the next stages of our work.

5.3 Exploring Cloud LLM Deployment with Gemini

After facing significant limitations with the local deployment of LLaMA 2 via `llama.cpp`, we decided to adopt a cloud-based LLM that could natively support tool use, structured interactions, and agent-based workflows. Our final choice was **Google's Gemini 2.5 Flash**, accessed through the `google_genai` provider.

5.3.1 Setup Steps

The model was initialized using:

```
model = init_chat_model("gemini-2.5-flash", model_provider="google_genai")
```

To validate the connection and configuration of the LLM backend, we conducted a test conversation with `gemini-2.5-flash` deployed via the LangChain integration. The model responded correctly to multiple prompts demonstrating that the setup was functional and that the server could successfully handle interactive requests. As shown in Figure 5.2, this interaction confirms the viability of the infrastructure for integrating a cloud-based LLM into the genomic gap-filling pipeline.

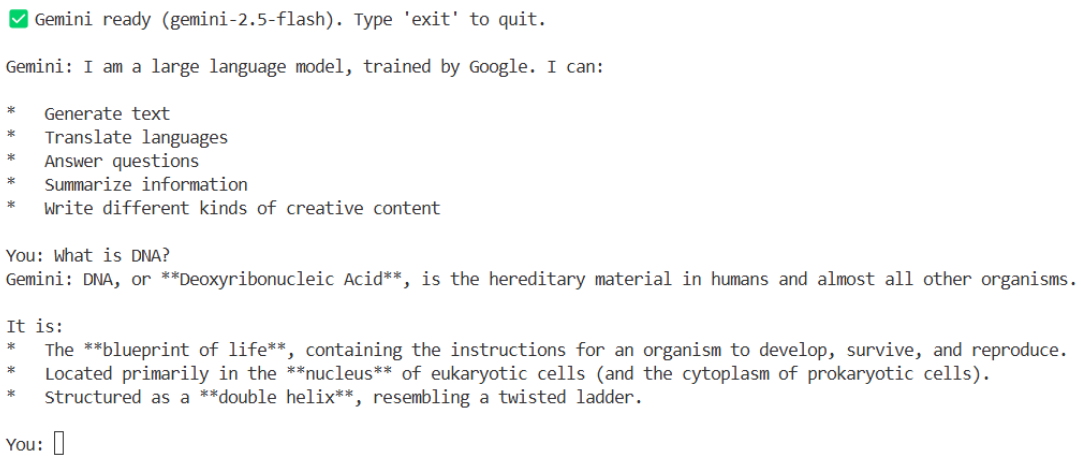

Figure 5.2: Example of a test conversation with `gemini-2.5-flash`, demonstrating correct setup and interactive capabilities.

5.3.2 Outcomes and Limitations

This version of Gemini was selected due to several key factors [54]:

- **Native tool support:** Gemini allows direct integration of tools (also called functions) into the agent pipeline, enabling the model to decide when and how to call external code.
- **High-speed inference:** the Flash variant is optimized for faster responses, ideal for iterative agent planning and tool execution loops.
- **Robust agent compatibility:** Gemini integrates seamlessly with LangChain's tool-calling agents, making it suitable for complex, multi-step reasoning over genomic data.

- **Modern instruction tuning:** unlike LLaMA, Gemini is instruction-finetuned to follow structured prompts, maintain conversational context, and return predictable outputs.

Although we originally prioritized local inference for reasons of privacy, control, and reproducibility, the practical requirements of building a functional tool-augmented agent led us to favor Gemini. The ability to embed custom tools, chain multiple steps, and handle structured data was indispensable for the DNA gap prediction pipeline.

However, several limitations were identified during this process. First, `gemini-2.5-flash` is a general-purpose model and thus inherits inherent usage constraints, such as rate limits, latency introduced by network communication, and restrictions on processing highly domain-specific scientific data. Additionally, while the model can accept relatively long input contexts, our experiments revealed performance degradation when entire genome sequences were introduced as a single prompt. In such cases, token limits and context window constraints necessitated pre-processing steps, including fragmenting genomic data before submission. This approach ensures compatibility with the model's maximum context length, but it introduces an additional loss of information.

5.4 Agent-Based Pipeline for Genomic Gap Prediction

In this section, we describe the complete implementation of an agent-based pipeline for genomic gap filling, designed using LangChain's tool-calling architecture. This system combines RAG, transformer-based MLM, and structured agent reasoning to iteratively solve DNA sequence completion tasks.

The motivation for adopting an agent-based approach lies in the proven strengths of LLM-driven agents in reasoning, planning, and interacting with complex environments [55]. Unlike traditional monolithic inference pipelines, LLM-based agents can be decomposed into specialized components, each with a distinct role, enabling modular decision-making and adaptive workflows. Multi-agent systems (MAS) extend this paradigm by allowing multiple specialized agents to collaborate, exchange information, and evolve their strategies over time. This collective intelligence is particularly valuable for tasks like genomic gap filling, where diverse subtasks(such as context retrieval, sequence generation and validation) can be distributed among cooperating agents.

5.4.1 Overview and Architecture

To better illustrate the agent-based approach adopted in our pipeline, we present an architectural overview of the system. The agent (planner) receives a genomic query from the user, plans a multi-step reasoning process, invokes two key tools,`context_tool` and `gap_filler_tool`, and uses an LLM to formulate a response.

The two main tools of the pipeline are the following:

- **Context Tool**: retrieves relevant genomic context sequences from a custom RAG corpus.
- **Gap Filler Tool**: completes the masked DNA sequence using a pretrained masked language model (GENA-LM).

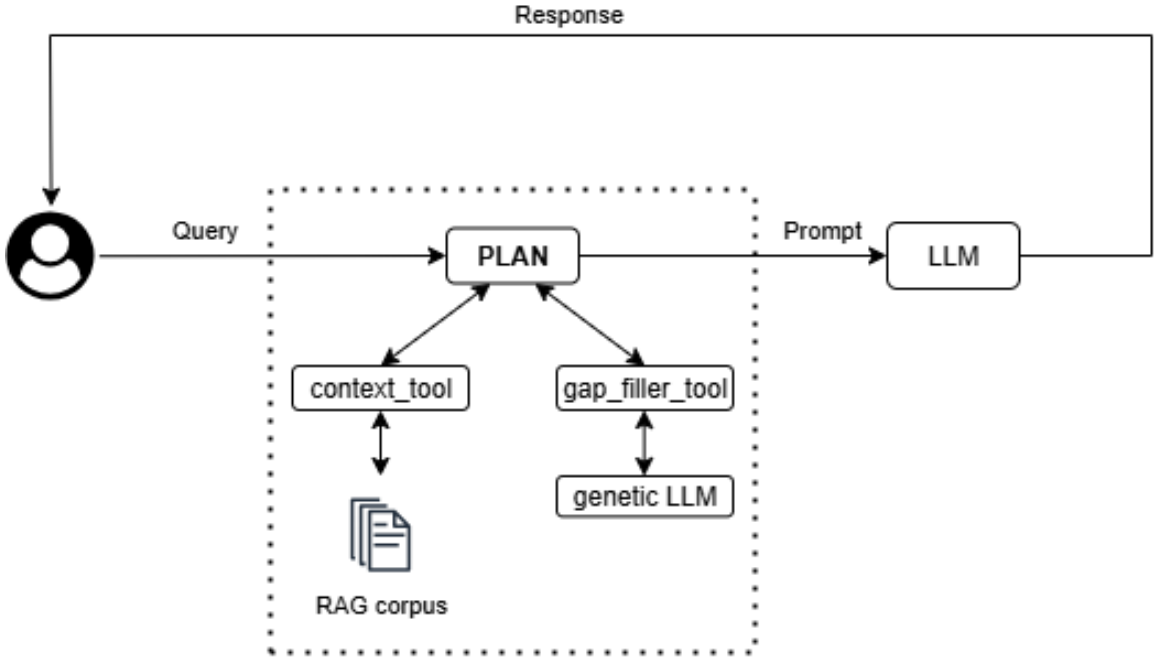

Figure 5.3: Agent architecture with integrated tools for gap filling.

These tools are orchestrated by a LangChain agent defined in `planner.py`, guided by a system prompt and executed with support from the selected LLM.

5.4.2 Main Execution Logic: `planner.py`

The core entrypoint of the pipeline is the `plan(user_input)` function:

- A system prompt instructs the agent to follow a strict two-step procedure: (1) use the context tool, then (2) fill the gaps.
- The prompt is wrapped using LangChain's `ChatPromptTemplate` and passed to a tool-calling agent.
- The LLM backend can be either local or cloud-based. For the cloud option, we use Google's `gemini-2.5-flash`, initialized via:

  ```
  model = init_chat_model("gemini-2.5-flash", model_provider=

  "google_genai")
  ```

  For local execution, we deploy `llama.cpp` using the official GGML Docker image with a pre-downloaded `llama-2-7b-chat.Q5_K_M.gguf` model.
- The agent is executed through an `AgentExecutor`, which invokes tools dynamically based on the LLM's responses.

5.4.3 Contextual Retrieval: `rag/retriever.py`

This module implements logic to load a directory of `.fna` files and match relevant sequences with the fragments flanking the gap.

- The function `load_all_sequences()` parses FASTA-style genomic files into a list of `(meta, sequence)` dictionaries.
- The function `retrieve_context(user_input)`:
  - Extracts the flanking fragments from the gapped sequence.
  - Searches for exact and partial matches in the corpus.
  - Returns the top 3 matches, including metadata headers, as a formatted string.

5.4.4 MLM-Based Gap Completion: `rag/gap_filler.py`

This module leverages the `GENA-LM BigBird` model, which supports long sequences (up to 4096 tokens) and is suitable for genomic data.

- **Model setup**:

```
from transformers import AutoTokenizer,
   BigBirdForMaskedLM
model = BigBirdForMaskedLM.from_pretrained("AIRI-
   Institute/gena-lm-bigbird-base-t2t")
```

- **Masked input construction**:
  - The function `build_masked_input_with_context()` scans a gapped DNA sequence (e.g., `ATGG---TTCA---GGA`) and adds reliable context around each gap using multiple retrieved RAG matches.
  - It first splits the sequence into fragments (delimited by "`---`") and for each gap, it attempts to extract shared substrings to the left and right from at least 2 matching sequences.
  - If consistent context is found on both sides, it is inserted before/after a `[MASKS]` placeholder to guide the language model.
  - **Example**:

```
Input sequence:    AGG---TCC
RAG matches:       ["AGGAATGGTCC", "AGGAATAGTCC", "AGGAATCGTCC"]

→ Left fragment:   AGG
→ Right fragment:  TCC

→ Common left context:  AAT (appears after AGG)
→ Common right context: G (appears before TCC)

→ Final masked input: AGGAAT [MASKS] GTCC
```

  - The function returns:
    - A masked input string for GENA-LM.
    - The number of nucleotides inferred and added as context.
- **Prediction logic**:

  - `predict_until_length()` attempts to predict a sequence of tokens that matches the desired gap length, iteratively adjusting the number of masks.
  - Predictions are decoded and filtered, and the closest-length result is returned.

5.4.5 Tool Wrappers: `tools/planner_tools.py`

LangChain `@tool` decorators are used to expose both tools to the agent:

- `context_tool(dna: str)`: wraps the RAG retrieval function.
- `gap_filler_tool(sequence: str, rag_matches: list[str], gap_length: int)`:
  - Constructs masked input from the gap and RAG context.
  - Adjusts for additional context length.
  - Generates the completed sequence using GENA-LM.

5.4.6 Pipeline Execution Flow

Once a DNA sequence with gaps is passed to the `plan()` function, the following steps occur:

1. The LLM reads the system prompt and invokes `context_tool` with the user input.
2. The tool returns up to 3 contextual matches from the RAG corpus.
3. The LLM next calls `gap_filler_tool` with:
   - The original sequence.
   - The retrieved RAG matches.
   - The expected gap length.
4. The final output is a DNA sequence with all gaps filled using context-aware predictions.

6. Conclusions

This section synthesizes the main findings obtained throughout the project, with a focus on the performance of various models in genomic gap prediction tasks, the improvements introduced by RAG-based augmentation, and the final implementation of an agent-based pipeline. The evaluation metrics presented in Chapters 3 and 4 are now analyzed in a comparative and integrated manner to assess the overall effectiveness of the approaches developed.

6.1 Symmary of Model Evaluation

This subsection synthesizes the quantitative results obtained in Chapter 3, where multiple genomic language models were evaluated under various conditions. The evaluation covered several metrics, including Top-60 Accuracy, Precision, Recall, F1 Score, Disk and RAM usage, Processing Time, and Levenshtein Distance, across different input sequence lengths (1000 bp, 2000 bp, and 5000 bp) and sequence coverage levels (10%–100%).

Among all models, `gena-lm-bigbird-base-t2t` consistently outperformed its counterparts across all metrics and input lengths. For instance, in **Top-60 Accuracy**, it started at approximately 0.85 for 1000 bp at 10% coverage and remained above 0.78 at full coverage. At 2000 bp, it maintained values above 0.85 across all coverage levels, and at 5000 bp it stayed consistently above 0.87, while many competing models dropped to zero due to token limitations.

In terms of **Precision**, `gena-lm-bigbird-base-t2t` achieved around 0.87 at 10% coverage for 1000 bp inputs, staying above 0.85 throughout. At 2000 bp it exceeded 0.90 for all coverage levels, and at 5000 bp it was the only model to surpass 0.92, with a slight upward trend as more context was provided. Most other models, such as `gena-lm-bert-base-t2t` and GROVER, began with moderate precision ($\sim$ 0.64–0.65 at 10% coverage for 1000 bp) but fell sharply (down to $\sim$ 0.48 or less) as coverage increased.

Recall results mirrored this pattern. At 5000 bp, `gena-lm-bigbird-base-t2t` maintained exceptional recall between 0.88 and 0.90 across all coverage levels, far exceeding the next best model (`nucleotide-transformer-v2-250m-multi-species`, $\sim$ 0.30). Even at shorter lengths, `gena-lm-bigbird-base-t2t` consistently held a $\geq$ 0.70 recall, compared to drops below 0.40 for most other models.

F1 Scores confirmed its balanced performance: at 1000 bp, `gena-lm-bigbird-base-t2t` began near 0.80 at 10% coverage and stayed above 0.70 at full coverage. At 2000 bp, it exceeded 0.85 across all levels, and at 5000 bp it maintained $\sim$ 0.90, while most other models were absent due to token limits or remained below 0.30.

From a resource usage perspective, `gena-lm-bigbird-base-t2t` required more **RAM** and **processing time** than smaller models. RAM consumption peaked at ~ 0.016 GB for 1000 bp and remained high for 5000 bp because it fully processed all tokens without truncation. In **processing time**, it reached nearly 1200 s per file at 5000 bp, comparable to the `nucleotide-transformer-v2-250m` but far slower than DNABERT-2 (< 5 s at 5000 bp) or GROVER. However, this additional cost translated directly into superior accuracy.

For **disk usage**, `gena-lm-bigbird-base-t2t` stayed moderate (~ 0.001 GB at 1000 bp) and near zero for longer contexts, while some other models like `nucleotide-transformer-v2-250m` exceeded 0.07 GB at 1000 bp.

Finally, in **Levenshtein Distance**, `gena-lm-bigbird-base-t2t` achieved the lowest distances across all input lengths and coverage levels, indicating the highest sequence fidelity. For example, at 5000 bp it outperformed the next viable model (`nucleotide-transformer-v2-250m`) by a large margin, producing outputs far closer to the ground truth.

Given this consistent numerical superiority, while all other models failed or degraded sharply, `gena-lm-bigbird-base-t2t` was ultimately selected as the final model for downstream gap prediction, as further justified in Section 3.4.

6.2 Impact of RAG on Gap Prediction

To address the limitations observed in the baseline gap prediction pipeline applied to simulated draft genomes, we explored the integration of RAG into the gap prediction pipeline. The goal was to enhance prediction performance by enriching the input with external genomic sequences that share similarity with the gap context, retrieved from a corpus built using real genomic data.

The RAG corpus was constructed through species-level annotation of simulated contigs, followed by automated downloading and indexing of relevant genomes. Using a dense vector index created with the FAISS library, semantically similar sequences were retrieved and appended to the model input in a structured format. These retrieved chunks were expected to provide additional biological context, potentially guiding the model toward more accurate predictions.

Despite the conceptual appeal, the experimental results showed that the introduction of RAG did not lead to consistent improvements in sequence identity. The identity percentages achieved with RAG were often slightly lower than those obtained without it. A likely explanation for this behavior lies in the architectural nature of the underlying model. The selected model, `gena-lm-bigbird-base-t2t`, is a MLM model, which does not natively incorporate mechanisms to differentiate or prioritize external context appended to the input string. As a result, the model treats retrieved sequences as regular tokens rather than as structured auxiliary information. This contrasts with autoregressive models, which are better equipped to condition their outputs on long, external prompts.

Consequently, although technically successful, this form of RAG integration was not effective in improving prediction accuracy within the current architecture. Future efforts may explore the use of decoder-only or encoder-decoder architectures, or alternatively, agent-based approaches that enable more flexible and structured use of retrieved context.

6.3 Performance of the Agent-Based Pipeline

Building upon the limitations identified in the RAG-enhanced approach, a novel agent-based pipeline was developed to improve genomic gap prediction. This architecture aimed to address the need for modularity, reasoning capabilities, and structured interaction with external tools and retrieval components.

The agent-based system was composed of several coordinated modules, each responsible for a distinct aspect of the gap prediction process. These included:

- **planner.py:** orchestrates the overall pipeline execution, determining the sequence of actions needed for each gap.
- **retriever.py:** performs vector-based semantic search over the FAISS index to fetch contextually relevant genomic sequences.
- **gap_filler.py:** applies the genomic model to generate predictions based on both local and retrieved context.
- **planner_tools.py:** implements specialized utility functions such as alignment scoring, token balancing, and format conversions.

The agent and tools communicate through well-defined prompts and outputs, enabling dynamic and flexible control over prediction tasks. This modular design facilitates experimentation, debugging, and future extensibility.

The agent pipeline successfully executed end-to-end predictions on multiple simulated gaps. The tasks were performed autonomously, and the interaction flow between components was stable and reproducible. By separating retrieval and generation, the system gained interpretability and ease of control. This architecture also opens the door to future improvements.

In summary, the agent-based pipeline proved to be a flexible, interpretable, and modular alternative to monolithic prediction systems. Although its current performance is constrained by model architecture, it represents a significant step toward intelligent, context-aware genomic gap prediction.

6.4 Limitations and Observed Bottlenecks

Despite the improvements achieved, several limitations were observed:

- **Low Identity Scores:** even with RAG, identity scores remained below 60% for many gaps, suggesting limits in the model's ability to reconstruct longer or more variable sequences.
- **Resource Usage:** high RAM and disk space requirements were encountered during genome indexing and retrieval, especially when handling large species datasets.
- **Tokenization and Sampling Challenges:** balancing the number of `[MASK]` tokens and sampling thresholds required iterative tuning.

7. Future Work

The present work represents only the first step toward building a fully capable, flexible, and efficient system for genomic gap-filling. While the pipeline developed here establishes a solid foundation, there is still significant room for exploration and enhancement. The current implementation focuses primarily on masked language models, evaluated under a specific experimental setup. Extending this framework to include autoregressive models would open new possibilities, enabling direct sequence generation without the constraints imposed by masking strategies and potentially improving both flexibility and predictive accuracy. In such an extension, the same systematic procedure for model evaluation and selection used in this work would be applied, ensuring that performance comparisons remain consistent and that the most suitable architectures for the gap-filling task are chosen based on rigorous, reproducible benchmarks.

The current retrieval-augmented generation RAG module operates exclusively on a small, static, and fully local corpus. While this setup simplifies development and avoids external dependencies, it presents a major limitation: the local dataset often does not contain any sequences that are relevant or sufficiently similar to the flanking regions of the gaps being filled. As a consequence, in many cases the retrieval step yields no useful matches, forcing the language model to attempt gap completion without meaningful contextual support, which negatively impacts the biological plausibility of the generated sequences.

A natural direction for future work is to expand the retrieval process beyond the local corpus by enabling access to large-scale, publicly available genomic repositories via the internet. This would allow the system to retrieve candidate sequences from a much broader search space, increasing the probability of finding relevant homologous or syntenic regions that match the flanking sequences surrounding the gap. Once retrieved, these sequences could be stored and indexed using a high-performance vector similarity search library such as FAISS. By embedding the sequences (for example, using fixed-length k-mer embeddings or learned representations from a DNA-specific encoder) and performing approximate nearest-neighbor search, FAISS would make it feasible to handle millions of reference sequences while keeping retrieval latency low [56].

This enhancement would transform the RAG component from a closed, locally constrained search into an open, internet-scale retrieval system. Such a system would be capable of dynamically incorporating the latest publicly available genomic data, thereby increasing robustness to novel or rare sequences. In practice, this could lead to gap completions that are not only syntactically valid in terms of nucleotide composition, but also grounded in real biological evidence retrieved from a diverse set of genomic sources.

From a computational perspective, the system currently operates under hardware limitations that restrict both the selection of models and the speed of inference. Deploying the pipeline on more powerful hardware, such as NVIDIA A100 GPUs, would not only allow

compatibility with a wider range of models, but also accelerate experimentation, making large-scale evaluations feasible. This is particularly relevant for expanding the benchmarking process to larger test sets containing multiple gaps per sequence. Such evaluations would more accurately reflect real-world genomic assembly challenges, but they would require significant computational resources and potentially benefit from hosting the test datasets in the cloud to avoid exhausting local resources.

Another important limitation is the pipeline's dependency on knowing the gap length in advance, as this information is currently required to determine the number of `[MASK]` tokens to insert. Developing an approach capable of predicting gap content without prior knowledge of its size would make the system more robust and applicable to real sequencing data, where such information is often unavailable. This would represent a major step toward closing the gap between controlled experiments and practical use cases.

Another promising avenue is the integration of the Model Context Protocol (MCP) to standardize and extend the way external data sources, tools, and computational resources are connected to the gap-filling pipeline [57]. By adopting MCP, the system could dynamically query and incorporate context from heterogeneous sources, including genomic databases, cloud storage systems, and specialized bioinformatics services, without requiring custom integration for each. This would make it possible to seamlessly combine local and remote resources, keep context windows synchronized across model interactions, and simplify the orchestration of multi-step reasoning processes. In the context of RAG, MCP could act as a unifying layer, enabling the retrieval module to incorporate diverse, up-to-date evidence into the generation process while preserving reproducibility and interoperability across different environments.

In addition to improving predictive capabilities, there is also potential to make the system more practical and accessible for integration into laboratory workflows. Implementing a REST API for model inference would enable researchers to interact with the pipeline through standardized endpoints, simplifying deployment and facilitating adoption. Furthermore, the agent-based architecture could be enriched with specialized agents, such as a judge component capable of selecting the most plausible gap completion among multiple candidate predictions. This multi-agent strategy could increase reliability and adaptability across diverse genomic contexts.

Taken together, these directions point toward a clear path for transforming the current prototype into a scalable, high-performance, and user-friendly tool that can support real-world genomic assembly efforts. The foundations laid in this work open the door to a broad range of improvements that, once implemented, could make the pipeline an indispensable asset for both research and applied genomics.

Bibliography

[1] Jiajia Liu, Mengyuan Yang, Yankai Yu, Haixia Xu, Tiangang Wang, Kang Li, and Xiaobo Zhou. Advancing bioinformatics with large language models: components, applications and perspectives. *arXiv preprint arXiv:2401.04155*, 2024.

[2] The Royal Swedish Academy of Sciences. The nobel prize in chemistry 2024. https://www.nobelprize.org/prizes/chemistry/2024/press-release/, 2024.

[3] Sriram Kosuri and George M Church. Large-scale de novo dna synthesis: technologies and applications. *Nature Methods*, 11(5):499–507, 2014.

[4] HBC Bioinformatics Training. Accessing genome reference data. https://hbctraining.github.io/Accessing_public_genomic_data/lessons/accessing_genome_reference_data.html, 2020. Accessed: 2024-05-12.

[5] Subir Verma. Retrieval-augmented generation — deep dive into its components. https://subirverma.medium.com/retrieval-augmented-generation-deep-dive-8e8db427709f, 2023.

[6] Shanghua Gao, Ada Fang, Yepeng Huang, Valentina Giunchiglia, Ayush Noori, Jonathan Richard Schwarz, Yasha Ektefaie, Jovana Kondic, and Marinka Zitnik. Empowering biomedical discovery with ai agents. *Cell*, 187(22):6125–6151, 2024.

[7] Zhi Jing, Yongye Su, and Yikun Han. When large language models meet vector databases: A survey. In *2025 Conference on Artificial Intelligence x Multimedia (AIxMM)*, pages 7–13, 2025.

[8] Fei Guo, Renchu Guan, Yaohang Li, Qi Liu, Xiaowo Wang, Can Yang, and Jianxin Wang. Foundation models in bioinformatics. *National Science Review*, 12(4):nwaf028, 01 2025.

[9] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences, 2021.

[10] Julia Zimmermann, Christoph Kaleta, and Silvio Waschina. gapseq: informed prediction of bacterial metabolic pathways and reconstruction of accurate metabolic models. *Genome Biology*, 22:81, 2021. Published 10 March 2021.

[11] Eric Chen, Justin Chu, Jessica Zhang, Rene L. Warren, and Inanc Birol. Gappredict – a language model for resolving gaps in draft genome assemblies. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 18(6):2802–2808, November 2021.

[12] Shentong Mo, Xi Fu, Chenyang Hong, Yizhen Chen, Yuxuan Zheng, Xiangru Tang, Zhiqiang Shen, Eric P Xing, and Yanyan Lan. Multi-modal self-supervised pre-training for regulatory genome across cell types, 2021.

[13] Ž. Avsec, V. Agarwal, D. Visentin, et al. Effective gene expression prediction from sequence by integrating long-range interactions. *Nature Methods*, 18:1196–1203, October 2021. Published: 04 October 2021; Accepted: 27 July 2021; Received: 17 February 2021.

[14] Meredith V. Trotter, Cuong Q. Nguyen, Stephen Young, Rob T. Woodruff, and Kim M. Branson. Epigenomic language models powered by cerebras, 2021.

[15] A. Hoarfrost, A. Aptekmann, G. Farfañuk, et al. Deep learning of a bacterial and archaeal universal language of life enables transfer learning and illuminates microbial dark matter. *Nature Communications*, 13:2606, May 2022. Published: 11 May 2022; Accepted: 30 March 2022; Received: 20 January 2021.

[16] Sumit Tarafder, Mazharul Islam, Swakkhar Shatabda, and Atif Rahman. Figbird: a probabilistic method for filling gaps in genome assemblies. *Bioinformatics*, 38(15):3717–3724, 06 2022.

[17] Ho-Jin Gwak and Mina Rho. Vibe: a hierarchical bert model to identify eukaryotic viruses using metagenome sequencing data. *Briefings in Bioinformatics*, 23(4):bbac204, 06 2022.

[18] Meng Yang, Lichao Huang, Haiping Huang, Hui Tang, Nan Zhang, Huanming Yang, Jihong Wu, and Feng Mu. Integrating convolution and self-attention improves language model of human genome for interpreting non-coding regions at base-resolution. *Nucleic Acids Research*, 50(14):e81–e81, 05 2022.

[19] Benjamin Levy, Zihao Xu, Liyang Zhao, Karl Kremling, Ross Altman, Phoebe Wong, and Chris Tanner. Florabert: cross-species transfer learning withattention-based neural networks for geneexpression prediction, 08 2022.

[20] Zeheng Bai, Yao-zhong Zhang, Satoru Miyano, Rui Yamaguchi, Kosuke Fujimoto, Satoshi Uematsu, and Seiya Imoto. Identification of bacteriophage genome sequences with representation learning. *Bioinformatics*, 38(18):4264–4270, 08 2022.

[21] Maxim Zvyagin, Alexander Brace, Kyle Hippe, Yuntian Deng, Bin Zhang, Cindy Orozco Bohorquez, Austin Clyde, Bharat Kale, Danilo Perez-Rivera, Heng Ma, Carla M. Mann, Michael Irvin, J. Gregory Pauloski, Logan Ward, Valerie Hayot-Sasson, Murali Emani, Sam Foreman, Zhen Xie, Diangen Lin, Maulik Shukla, Weili Nie, Josh Romero, Christian Dallago, Arash Vahdat, Chaowei Xiao, Thomas Gibbs, Ian Foster, James J. Davis, Michael E. Papka, Thomas Brettin, Rick Stevens, Anima Anandkumar, Venkatram Vishwanath, and Arvind Ramanathan. Genslms: Genome-scale language models reveal sars-cov-2 evolutionary dynamics. *bioRxiv*, 2022.

[22] C. V. Theodoris, L. Xiao, A. Chopra, et al. Transfer learning enables predictions in network biology. *Nature*, 618:616–624, June 2023. Published: 31 May 2023; Accepted: 27 April 2023; Received: 29 March 2022.

[23] Gonzalo Benegas, Sanjit Singh Batra, and Yun S. Song. Dna language models are powerful zero-shot predictors of genome-wide variant effects. *bioRxiv*, 2023.

[24] Daoan Zhang, Weitong Zhang, Bing He, Jianguo Zhang, Chenchen Qin, and Jianhua Yao. Dnagpt: A generalized pretrained tool for multiple dna sequence analysis tasks. *bioRxiv*, 2023.

[25] Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Callum Birch-Sykes, Michael Wornow, Aman Patel, Clayton Rabideau, Stefano Massaroli, Yoshua Bengio, Stefano Ermon, Stephen A. Baccus, and Chris Ré. Hyenadna: Long-range genomic sequence modeling at single nucleotide resolution, 2023.

[26] Bin Shao and Jiawei Yan. A long-context language model for deciphering and generating bacteriophage genomes. *Nature Communications*, 15(1):9392, 2024.

[27] Gonzalo Benegas, Carlos Albors, Alan J. Aw, Chengzhong Ye, and Yun S. Song. Gpn-msa: an alignment-based dna language model for genome-wide variant effect prediction. *bioRxiv*, 2023.

[28] Yanyi Chu, Dan Yu, Yupeng Li, Kaixuan Huang, Yue Shen, Le Cong, Jason Zhang, and Mengdi Wang. A 5' utr language model for decoding untranslated regions of mrna and function predictions. *bioRxiv*, 2023.

[29] Y. Chen, G. Wang, and T. Zhang. Utilizing deep neural networks to fill gaps in small genomes. *International Journal of Molecular Sciences*, 25(15):8502, August 2024.

[30] Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu. Dnabert-2: Efficient foundation model and benchmark for multi-species genome, 2024.

[31] Bernardo P. de Almeida, Hugo Dalla-Torre, Guillaume Richard, Christopher Blum, Lorenz Hexemer, Maxence Gélard, Priyanka Pandey, Stefan Laurent, Alexandre Laterre, Maren Lang, Uğur Şahin, Karim Beguir, and Thomas Pierrot. Segmentnt: Annotating the genome at single-nucleotide resolution with dna foundation models. *bioRxiv*, 2024. Preprint, Cold Spring Harbor Laboratory.

[32] Yair Schiff, Chia-Hsiang Kao, Aaron Gokaslan, Tri Dao, Albert Gu, and Volodymyr Kuleshov. Caduceus: Bi-directional equivariant long-range dna sequence modeling, 2024.

[33] J. Mendoza-Revilla, E. Trop, L. Gonzalez, et al. A foundational large language model for edible plant genomes. *Communications Biology*, 7:835, July 2024. Published: 09 July 2024; Accepted: 17 June 2024; Received: 21 November 2023.

[34] M. Sanabria, J. Hirsch, P.M. Joubert, et al. Dna language model grover learns sequence context in the human genome. *Nature Machine Intelligence*, 6:911–923, August 2024. Published: 23 July 2024; Accepted: 26 June 2024; Received: 31 August 2023.

[35] Qihang Zhao, Chi Zhang, and Weixiong Zhang. dnagrinder: a lightweight and high-capacity genomic foundation model, 2024.

[36] Huaqing Liu, Shuxian Zhou, Peiyi Chen, Jiahui Liu, Ku-Geng Huo, and Lanqing Han. Exploring genomic large language models: Bridging the gap between natural language and gene sequences. *bioRxiv*, 2024.

[37] Zhiwei Gao, Qiang Liu, Weijie Zeng, et al. Epigept: a pretrained transformer-based language model for context-specific human epigenomics. *Genome Biology*, 25:310, 2024. Published 18 December 2024.

[38] Ken Chen, Yue Zhou, Maolin Ding, Yu Wang, Zhixiang Ren, and Yuedong Yang. Self-supervised learning on millions of primary rna sequences from 72 vertebrates improves

sequence-based rna splicing prediction. *Briefings in Bioinformatics*, 25(3):bbae163, 04 2024.

[39] Eric Nguyen, Michael Poli, Matthew G. Durrant, Armin W. Thomas, Brian Kang, Jeremy Sullivan, Madelena Y. Ng, Ashley Lewis, Aman Patel, Aaron Lou, Stefano Ermon, Stephen A. Baccus, Tina Hernandez-Boussard, Christopher Ré, Patrick D. Hsu, and Brian L. Hie. Sequence modeling and design from molecular to genome scale with evo. *bioRxiv*, 2024.

[40] Wang Liang. Llama-gene: A general-purpose gene task large language model based on instruction fine-tuning, 2024.

[41] Veniamin Fishman, Yuri Kuratov, Aleksei Shmelev, Maxim Petrov, Dmitry Penzar, Denis Shepelin, Nikolay Chekanov, Olga Kardymon, and Mikhail Burtsev. Gena-lm: a family of open-source foundational dna language models for long sequences. *Nucleic Acids Research*, 53(2):gkae1310, 01 2025.

[42] Monireh Safari, Pablo Millan Arias, Scott C. Lowe, Lila Kari, Angel X. Chang, and Graham W. Taylor. Enhancing dna foundation models to address masking inefficiencies. *arXiv preprint arXiv:2502.18405*, 2025. arXiv:2502.18405v1 [cs.LG], Joint first authors: Safari and Millan Arias. Corresponding authors: gwtaylor@uguelph.ca, lila@uwaterloo.ca.

[43] H. Dalla-Torre, L. Gonzalez, J. Mendoza-Revilla, et al. Nucleotide transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 22:287–297, February 2025. Published: 28 November 2024; Accepted: 19 October 2024; Received: 01 March 2023.

[44] Li Yujian and Liu Bo. A normalized levenshtein distance metric. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(6):1091–1095, 2007.

[45] Muhammad Arslan, Hussam Ghanem, Saba Munawar, and Christophe Cruz. A survey on rag with llms. *Procedia computer science*, 246:3781–3790, 2024.

[46] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *CoRR*, abs/2005.11401, 2020.

[47] Gustavo de Aquino e Aquino, Nádila da Silva de Azevedo, Leandro Youiti Silva Okimoto, Leonardo Yuto Suzuki Camelo, Hendrio Luis de Souza Bragança, Rubens Fernandes, Andre Printes, Fábio Cardoso, Raimundo Gomes, and Israel Gondres Torné. From rag to multi-agent systems: A survey of modern approaches in llm development. 2025.

[48] Yucheng Ding, Chaoyue Niu, Fan Wu, Shaojie Tang, Chengfei Lyu, and Guihai Chen. Enhancing on-device llm inference with historical cloud-based llm interactions. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '24, page 597–608, New York, NY, USA, 2024. Association for Computing Machinery.

[49] Peter Sandrini. Beyond the cloud: Assessing the benefits and drawbacks of local llm deployment for translators, 2025.

[50] Timur Isachenko and Shamim Bhuiyan. *Generative AI with local LLM: A comprehensive roadmap for building AI-Driven applications with local LLMs*. Leanpub, 2024.

[51] Teresa Peng, Liam Liu, Maya Gupta, Kabir Mehta, Aditya Nair, Saanvi Desai, and Priya Singh. Enhanced hybrid inference techniques for scalable on-device llm personalization and cloud integration. *contexts*, 8(17):18–25, 2024.

[52] Oguzhan Topsakal and Tahir Cetin Akinci. Creating large language model applications utilizing langchain: A primer on developing llm apps fast. In *International conference on applied engineering and natural sciences*, volume 1, pages 1050–1056, 2023.

[53] Laura Huber, Scott Michael, Jefferson Davis, and Abhinav Thota. Delivering large language model platforms with hpc.

[54] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, and Noveen Sachdeva. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025.

[55] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 1(1):9, 2024.

[56] B Sriman, SH Annie Silviya, Y Mouleesh, S Vinod, S Nishanthini, and Polimera Nikitha. Intelligent document interaction with advanced vector embeddings and faiss-cpu indexing. In *2024 8th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*, pages 1–6. IEEE, 2024.

[57] Raj Yavatkar. Mcp: A protocol for coordination and temporal synchronization in multimedia collaborative applications. In *1992 12th International Conference on Distributed Computing System*, pages 606–607. IEEE Computer Society, 1992.